Faculty of Informatics Engineering Department of Systems Security and Computer Networking

# RAG Based Security Auditor: Automated CVE and OWASP Top 10 Code Compliance Using Retrieval-Augmented Generation

Prepared by:

Nagham Ashqar

Moustafa Seifo

First release

2025-2026

# Abstract:

This project presents a Retrieval-Augmented Generation (RAG)-based security auditor designed to support automated source code vulnerability analysis and CWE/OWASP mapping. The proposed system addresses a key limitation of Large Language Model (LLM)-only security auditing: relying solely on the model's internal knowledge can lead to inconsistent vulnerability classification, limited traceability, and weak grounding in known security cases. To reduce this limitation, the system integrates semantic code retrieval with LLM-based analysis, allowing the model to reason over both the submitted code and relevant examples retrieved from a local vulnerability knowledge base.

The implemented prototype uses a SecureCode-derived dataset as its primary knowledge source. Each vulnerable code example is stored with security metadata, including CWE identifier, OWASP category, programming language, CVE reference when available, CVSS score, and incident description. During the offline ingestion phase, vulnerable code snippets are converted into semantic embeddings using the microsoft/unixcoder-base model and stored in a persistent ChromaDB vector database. During runtime, a submitted code snippet is embedded using the same UniXcoder model, and the system retrieves the top-k semantically similar vulnerable examples from the knowledge base. These retrieved examples are then combined with the submitted code in a structured prompt and passed to an OpenAI GPT model, which produces a JSON-based security analysis containing the predicted CWE, OWASP mapping, confidence score, explanation, and remediation guidance.

To evaluate the effectiveness of retrieval augmentation, the system was compared against a baseline LLM-only approach. A held-out test set of 50 vulnerable samples was constructed from the top five CWE categories in the dataset, with the selected samples removed from the knowledge base before rebuilding the vector database to avoid retrieval leakage. The evaluation measured CWE classification performance using Accuracy, Precision, Recall, and F1-score. The RAG-based approach improved the average F1-score from 0.434 for the baseline LLM to 0.793, indicating that semantic retrieval of similar vulnerable examples improves the model's ability to classify vulnerability categories.

The current prototype focuses on snippet-level vulnerability auditing and CWE classification. Future enhancements include developing a command-line interface for directory-level auditing, enriching the knowledge base with secure code implementations and CVE descriptions, adding semantic description-based retrieval, implementing chunking strategies for long source files, and expanding the evaluation to include safe-versus-vulnerable detection. Overall, the project demonstrates that combining code-specific embeddings, vector-based retrieval, vulnerability metadata, and structured LLM output provides a practical and explainable foundation for AI-assisted security auditing.



Keywords: Retrieval-Augmented Generation (RAG), Code Embeddings, Secure Code Analysis.

# الملخص :

يعرض هذا المشروع مدققًا أمنيًا قائمًا على التوليد المعزز بالاسترجاع (Retrieval-Augmented Generation - RAG)، ومصممًا لدعم التحليل الآلي لثغرات الشيفرة المصدرية وربطها بتصنيفات CWE وOWASP. يعالج النظام المقترح أحد القيود الرئيسية في التدقيق الأمني المعتمد فقط على نماذج اللغة الكبيرة (LLM)، إذ إن الاعتماد فقط على المعرفة الداخلية للنموذج قد يؤدي إلى تصنيف غير متسق للثغرات، وضعف في إمكانية التتبع، وضعف في الارتكاز على حالات أمنية معروفة. ولتقليل هذا القيد، يدمج النظام بين الاسترجاع الدلالي للشيفرة والتحليل المعتمد على نموذج لغوي كبير، مما يسمح للنموذج بالاستدلال اعتمادًا على كلٍّ من الشيفرة المقدمة والأمثلة ذات الصلة المسترجعة من قاعدة معرفة محلية للثغرات.

يستخدم النموذج الأولي المنفذ مجموعة بيانات مشتقة من SecureCode كمصدر أساسي للمعرفة. يتم تخزين كل مثال لشيفرة ضعيفة أمنيًا مع بيانات وصفية أمنية، تتضمن معرف CWE، وفئة OWASP، ولغة البرمجة، ومرجع CVE عند توفره، ودرجة CVSS، ووصف الحادثة. خلال مرحلة الإدخال غير المتصلة بالإنترنت، يتم تحويل مقاطع الشيفرة الضعيفة أمنيًا إلى تضمينات دلالية باستخدام نموذج microsoft/unixcoder-base وتخزينها في قاعدة بيانات متجهية دائمة باستخدام ChromaDB. أثناء وقت التشغيل، يتم تضمين مقطع الشيفرة المقدم باستخدام نموذج UniXcoder نفسه، ويسترجع النظام أفضل k أمثلة ضعيفة أمنيًا متشابهة دلاليًا من قاعدة المعرفة. بعد ذلك، تُدمج هذه الأمثلة المسترجعة مع الشيفرة المقدمة ضمن موجه منظم وتُمرر إلى نموذج OpenAI GPT، والذي ينتج تحليلًا أمنيًا بصيغة JSON يحتوي على تصنيف CWE المتوقع، وربط OWASP، ودرجة الثقة، والتفسير، وإرشادات المعالجة.

لتقييم فعالية التعزيز بالاسترجاع، تمت مقارنة النظام مع نهج أساسي يعتمد فقط على نموذج LLM. تم إنشاء مجموعة اختبار محجوزة مكونة من 50 عينة ضعيفة أمنيًا من أكثر خمس فئات CWE تكرارًا في مجموعة البيانات، مع إزالة العينات المختارة من قاعدة المعرفة قبل إعادة بناء قاعدة البيانات المتجهية لتجنب تسرب الاسترجاع. قاس التقييم أداء تصنيف CWE باستخدام الدقة (Accuracy)، والدقة الإيجابية (Precision)، والاسترجاع (Recall)، ودرجة F1. حسّن النهج القائم على RAG متوسط درجة F1 من 0.434 في نموذج LLM الأساسي إلى 0.793، مما يشير إلى أن الاسترجاع الدلالي للأمثلة الضعيفة أمنيًا المتشابهة يحسن قدرة النموذج على تصنيف فئات الثغرات.

يركز النموذج الأولي الحالي على تدقيق الثغرات على مستوى مقاطع الشيفرة وتصنيف CWE. تشمل التحسينات المستقبلية تطوير واجهة سطر أوامر للتدقيق على مستوى المجلدات، وإثراء قاعدة المعرفة بتطبيقات الشيفرة الآمنة وأوصاف CVE، وإضافة الاسترجاع القائم على الوصف الدلالي، وتنفيذ استراتيجيات التجزئة لملفات الشيفرة الطويلة، وتوسيع التقييم ليشمل الكشف بين الشيفرة الآمنة والضعيفة أمنيًا. بشكل عام، يوضح المشروع أن الجمع بين التضمينات الخاصة بالشيفرة، والاسترجاع القائم على المتجهات، والبيانات الوصفية للثغرات، ومخرجات LLM المنظمة يوفر أساسًا عمليًا وقابلًا للتفسير للتدقيق الأمني المدعوم بالذكاء الاصطناعي.



كلمات مفتاحية : التوليد المعزز بالاسترجاع (RAG)، تضمينات الشيفرة، تحليل الشيفرة الآمن.

# Contents

## Figures

# Tables

# Chapter 1: Theoretical Background

## 1.1 Introduction

Software vulnerabilities represent a critical challenge in modern computing systems, posing significant threats to software reliability, data integrity, and overall system security. These vulnerabilities, often arising from programming errors, design flaws, or misconfigurations, can be exploited by malicious actors to cause severe consequences such as unauthorized access, data leakage, system disruption, and financial losses [1]. With the rapid growth in software complexity and scale, the number and diversity of vulnerabilities have increased substantially, making manual code auditing increasingly inefficient, time-consuming, and error-prone [2]. Consequently, automated vulnerability detection techniques have become essential for ensuring secure software development.

Traditional approaches to vulnerability detection, including rule-based and signature-based methods, rely heavily on predefined patterns and expert knowledge. While effective for known vulnerabilities, these techniques often fail to generalize to unseen or complex attack patterns and tend to suffer from high false positive and false negative rates [3]. To overcome these limitations, recent research has explored the use of machine learning and deep learning models, which aim to automatically learn vulnerability patterns from source code representations such as token sequences, abstract syntax trees, and program graphs. However, these approaches are frequently constrained by dataset quality issues, including reliance on synthetic datasets or noisy real-world samples, which limits their generalization capability in practical scenarios [3] .

More recently, the emergence of Large Language Models (LLMs) has introduced a paradigm shift in software vulnerability detection. Owing to their strong capabilities in code understanding, contextual reasoning, and pattern recognition, LLMs have demonstrated promising results in analyzing source code and identifying potential vulnerabilities [1] . These models enable vulnerability detection without requiring compilation or execution, thereby simplifying analysis workflows and improving adaptability across diverse programming environments [4]. Despite these advantages, several studies have revealed that LLM-based approaches still face fundamental limitations. In particular, LLMs often struggle to capture the root causes of vulnerabilities and tend to rely on superficial code features, leading to poor performance when distinguishing between highly similar vulnerable and patched code [5] .Additionally, LLMs are limited by their static knowledge, which cannot keep pace with the continuously evolving vulnerability landscape, further constraining their effectiveness in real-world applications [6].

To address these challenges, recent research has focused on enhancing LLM-based vulnerability detection through the integration of external knowledge and advanced reasoning mechanisms. Retrieval-Augmented Generation (RAG) has emerged as a promising approach that augments LLMs with domain-specific knowledge retrieved from external sources such as vulnerability databases and historical CVE records. By incorporating relevant contextual information during inference, RAG improves both the accuracy and robustness of vulnerability detection systems [1]. For instance, the VUL-RAG framework introduces a knowledge-level retrieval mechanism

that extracts high-level vulnerability knowledge, including functional semantics, root causes, and fixing strategies, enabling LLMs to better understand vulnerability behaviors and significantly improving detection performance [5].

In addition to knowledge integration, structured reasoning techniques such as Chain-of-Thought (CoT) prompting have been proposed to enhance the interpretability and consistency of LLM-based analysis. By guiding models through step-by-step reasoning processes, CoT enables more accurate assessment of vulnerability characteristics, including exploitability and impact, thereby improving decision-making quality [7] . The combination of RAG and CoT has demonstrated notable improvements in vulnerability assessment tasks, highlighting the importance of integrating external knowledge with structured reasoning.

Moreover, advanced architectures such as multi-agent systems have been proposed to further address the complexity of vulnerability detection. These approaches decompose the detection task into specialized components, allowing different agents to focus on specific vulnerability types while leveraging retrieval mechanisms to ground their reasoning. Such designs have shown improved performance in handling diverse vulnerability patterns and reducing hallucination issues in LLM-based systems [2]. However, despite these improvements, such multi-agent architectures introduce increased system complexity, higher computational overhead, and scalability challenges, which may limit their practicality in real-world deployment.

Based on these advancements, it is evident that combining LLMs with knowledge-driven retrieval mechanisms and structured reasoning strategies represents a promising direction for improving automated vulnerability detection. However, challenges related to knowledge representation, retrieval effectiveness, and semantic understanding of vulnerabilities remain open research problems.

Therefore, this study aims to investigate and develop an effective framework for automated software vulnerability detection by leveraging Retrieval-Augmented Generation and advanced code representation techniques. The proposed approach focuses on enhancing the semantic understanding of vulnerabilities, improving detection accuracy, and addressing the limitations of existing LLM-based methods through the integration of external knowledge and contextual reasoning.

In order to strengthen the argument for a RAG-based security auditor, this research consolidates existing studies across eight essential dimensions, each of which directly influences the design choices of our project. The review initiates by laying out the empirical and theoretical foundations of RAG in vulnerability detection, showcasing its advantages over other methods. Subsequently, it explores the development of knowledge base, the creation of code embeddings, retrieval systems, strategies for integrating LLMs, and multi-agent options, before wrapping up with assessment techniques and practical implementation factors.

## 1.2 Large Language Models

Language generation, as a sequential task, was historically addressed by recurrent architectures like Long Short-Term Memory (LSTM) networks. However, the sequential nature of these models hindered large-scale parallelization during training, limiting their capacity.

The introduction of the Transformer architecture marked a paradigm shift by replacing recurrence with self-attention mechanisms, enabling massive parallelization while preserving the ability to model sequential dependencies.

Subsequent exploration led to the dominance of the decoder-only architecture for generative tasks and the encoder-only architecture for language understanding tasks. Modern LLMs are typically pre-trained on vast, general-domain corpora using a next-token prediction objective which imbues them with broad, generalist capabilities.

Following pre-training, these models often undergo a post-training alignment phase. This includes Supervised Fine-Tuning (SFT) on high-quality instruction datasets to improve their ability to follow commands and Reinforcement Learning from Human Feedback (RLHF) to enhance safety and align their behavior with human preferences.

The resulting foundation models are capable of performing a wide array of downstream tasks with remarkable zero-shot or few-shot performance [8].

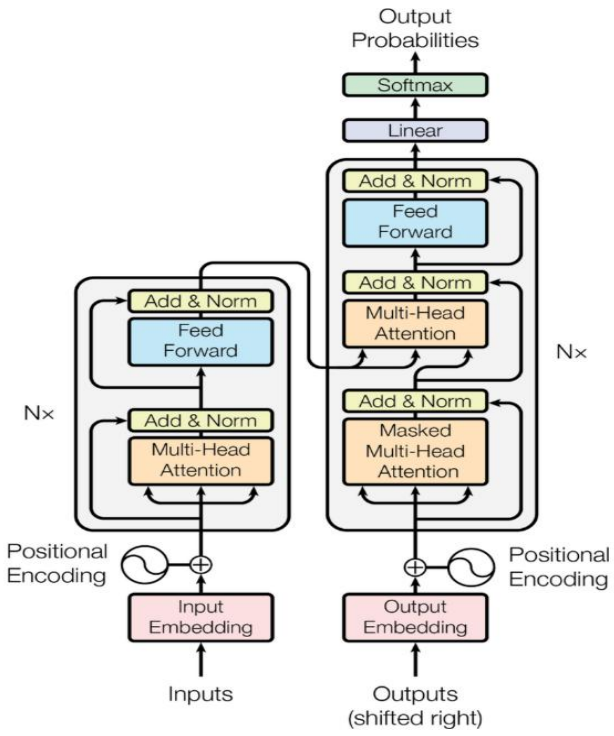


**Figure 1: Transformer's Architecture *[9].***

## 1.3 Retrieval Augmented Generation

LLMs often struggle with very specific topics, tending to produce hallucinations making it challenging to ensure factual correctness. This happens because the knowledge of the LLM is stored in its parameters (parametric memory), and it is highly compressed. Retrieval-Augmented Generation (RAG) tries to address this issue by incorporating a non-parametric memory, such as a database, to enhance the output's quality in knowledge-intensive tasks.

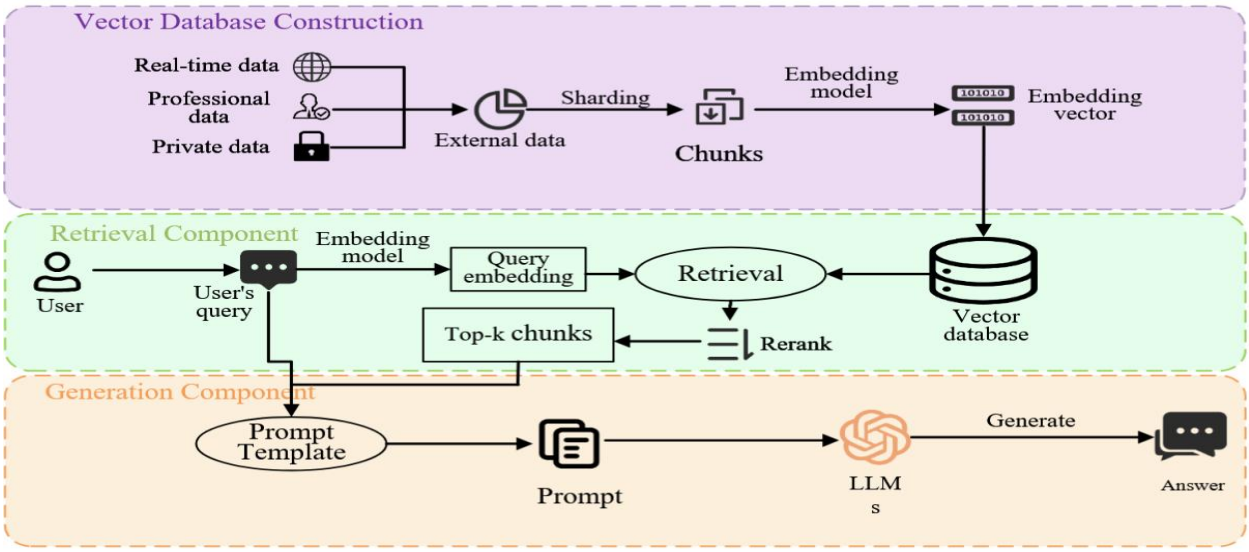


**Figure 2: RAG Technical Workflow *[35]*.**

RAG's key idea is to utilize external knowledge bases to fetch relevant information based on semantic similarity with the prompt, thereby reducing hallucinations. This approach has gained traction, making RAG essential in developing advanced chatbots. The usage of RAG is also extremely useful to connect private data, not present in the initial training data, to an LLM without requiring any fine-tuning procedure [9].

## 1.4 Fine-Tuning

Despite their scale, LLMs possess a finite information capacity constrained by their parameter count. This limitation means they cannot achieve optimal performance across all specialized domains, particularly those requiring deep, niche expertise. Consequently, fine-tuning remains a critical step to adapt these generalist models and enhance their performance on specific tasks.

However, fully fine-tuning an LLM, which involves updating all of its billions of parameters, is prohibitively expensive in terms of computational resources, memory, and time. This has led to the development of Parameter-Efficient Fine-Tuning (PEFT) methods, which aim to adapt LLMs by updating only a small fraction of their total parameters, thus making the fine-tuning process more accessible and efficient.

PEFT techniques can be broadly categorized into several approaches. One major branch of PEFT involves selective fine-tuning, where only a small, pre-defined subset of the model's original

parameters are updated. For instance, a common baseline approach is to freeze the entire model except for the final prediction layer.

A more sophisticated example is BitFit, which proposes fine-tuning only the bias terms of the model and leaving all other parameters frozen.

These methods are efficient as they don't introduce any new parameters and only require storing the updates for a small fraction of the existing ones. Other strategies focus on model compression. Knowledge distillation involves training a smaller student model to replicate the output distribution of a larger teacher model, effectively transferring the specialized knowledge into a more compact form.

Pruning techniques aim to reduce model size by identifying and removing redundant or unimportant weights from the network after fine-tuning. Among the most successful and widely adopted PEFT strategies are low-rank methods.

This approach is motivated by the observation that the weight updates during adaptation have a low intrinsic rank. Instead of modifying the full weight matrix, methods like Low-Rank Adaptation (LoRA) freeze the original model weights and inject trainable low-rank matrices into the Transformer layers. By decomposing the weight update into two smaller matrices, LoRA can achieve performance comparable to full fine-tuning while updating only a minuscule fraction (e.g., <0.01%) of the parameters.

This efficiency has made it a cornerstone of modern LLM adaptation. Building on this, subsequent work like QLoRA has combined low-rank adaptation with quantization techniques to further reduce memory usage, enabling the fine-tuning of massive models on a single GPU [8].

## 1.5 Multi-Agent Systems

LLM-based multi agent (MAS) are systems composed of multiple LLM-based agents that collaborate, communicate, and coordinate to accomplish complex tasks. The purpose of these systems is to simulate collective intelligence and mimic human-like social behavior through interaction, reasoning, planning, and adaptation [1].

## 1.6 The Security Crisis in LLM-Generated Code

AI coding assistants produce vulnerable code in 45% of generated implementations. Veracode's 2025 GenAI Code Security Report analyzed code from leading AI assistants and found nearly half of security-relevant implementations contained Common Weakness Enumeration (CWE) vulnerabilities.

This represents systematic risk in AI-assisted development, compounding security debt across millions of developers. The risk surface has scaled with adoption.

The issue goes beyond individual bugs. AI-generated vulnerabilities enter production codebases silently, without the traditional code review scrutiny applied to human-written code. Developers

trust AI assistants to produce functional implementations, but these tools lack security context to recognize when "functional" means "exploitable." Apiiro (2025) found AI copilots introduced 322% more privilege escalation paths and 153% more architectural design flaws compared to manually-written code, while generating 10× more security findings overall—actively degrading security practices. This creates a multiplier effect.

Vulnerable patterns suggested by AI assistants propagate across multiple projects. SQL injection flaws spread through microservices architectures. Authentication bypasses replicate across API endpoints. Cryptographic failures multiply through mobile applications.

The scale of AI adoption makes this a systematic risk to software security. LLMs trained on public code repositories learn from millions of vulnerable examples. Stack Overflow answers from 2010 showing insecure MySQL queries. GitHub repositories implementing broken authentication. Tutorial code demonstrating SQL injection vulnerabilities as "simple examples." These models learn what code looks like, not what secure code requires [10].

## 1.7 LLMs for Cybersecurity

Large language models have emerged as tools for cybersecurity automation, with recent surveys cataloging applications across vulnerability detection, malware analysis, threat intelligence, and incident response [11].

### 1.7.1 Limitations of LLM Only Approaches

Vul-RAG, a pioneering framework proposed by Du et al. [5]. was born from the observation that LLMs perform poorly (accuracy of only 0.06-0.14) at distinguishing between vulnerable code and its similar-but-patched counterpart, indicating a struggle to capture root causes.

LLMs, when used in isolation for vulnerability detection, exhibit significant limitations. They are prone to generating code with vulnerabilities and struggle to effectively generate security functions that precisely satisfy security requirements, an issue that likely stems from insufficient scrutiny of training data, lack of task-specific knowledge, and inadequate fine-tuning [9].

Moreover, most LLMs are trained on general-purpose corpora and thus lack the domain-specific knowledge essential for effective software vulnerability assessment, tending to rely on shallow pattern matching instead of deep contextual "reasoning" [7]. The comparative study by Saju et al. [1] .confirms this, showing a baseline LLM achieving an F1 score of only 0.67, which is substantially lower than RAG's 0.85. These limitations collectively underscore the necessity of a knowledge-grounded approach.

### 1.7.2 Effectiveness of RAG, Supervised Fine-Tuning, and Multi-Agents in Vulnerability Detection

An empirical evaluation by Saju et al. [1]. Provides a rigorous head-to-head comparison of three LLM-based vulnerability detection approaches against a baseline model. The study evaluated

RAG, Supervised Fine-Tuning (SFT), and a Dual-Agent framework on a curated dataset focusing on five critical CWE categories (CWE-119, CWE-399, CWE-264, CWE-20, and CWE-200) compiled from Big-Vul and real-world GitHub repositories.

The results are unequivocal: RAG achieved the highest overall accuracy of 0.86 and an F1 score of 0.85, outperforming both the SFT approach (which utilized parameter-efficient QLoRA adapters) and the Dual-Agent system (which employed a secondary agent for auditing and refining outputs).

The Dual-Agent system, while showing promise in improving “reasoning” transparency and error mitigation, did not match RAG's raw performance, highlighting that incorporating domain expertise via retrieval is a more significant performance lever than architectural complexity alone.

## 1.8 Knowledge Base Construction

### 1.8.1 Vulnerability Knowledge Structuring

The effectiveness of a RAG system is fundamentally tied to the quality and structure of its knowledge base.

Vul-RAG's first phase is dedicated to constructing this base by using an LLM to extract multi-dimensional knowledge from existing CVE instances. The key insight from Vul-RAG is that the knowledge must be high-level and generalizable, moving beyond simple code-near snippets to encompass functional semantics, root causes, and remediation strategies.

This approach ensures that the retrieved knowledge can be effectively applied to new, unseen code that shares semantic similarities with known vulnerabilities, rather than just lexical ones [5].

### 1.8.2 Vulnerability Representation

The representation of vulnerability knowledge directly impacts retrieval effectiveness. Vul-RAG's emphasis on high-level, generalizable knowledge is a direct response to the failure of lexical similarity to capture the subtle semantic differences between vulnerable and patched code. The knowledge extracted by Vul-RAG covering functional semantics, root causes, and fixing solutions—provides a more abstract and transferable representation than raw code alone. This allows the retriever to identify relevant knowledge based on the functional semantics of the code snippet under analysis, a more robust approach than relying on surface-level textual similarity [5].

### 1.8.3 Knowledge Base Security

A critical, often overlooked, aspect of knowledge base construction is security. The very mechanism that makes RAG powerful is the ingestion of external data which also creates a significant attack surface.

RAG systems retrieve documents from vector databases and send them directly to LLMs without verification, creating a vulnerability where a poisoned knowledge base can lead to the LLM generating dangerous advice.

Research by Habler et al. [12]. Introduces the concept of "hubness poisoning," where malicious actors can exploit hubs—items that frequently appear in top-k retrieval results for varied queries—to introduce harmful content, alter search rankings, or bypass content filtering.

Their open-source tool, HubScan, addresses this by providing a multi-detector architecture for evaluating vector indices and embeddings to identify hubs, supporting major vector databases like FAISS, Pinecone, Qdrant, and Weaviate.

The RAGShield project further exemplifies this concern, sitting between retrieval and generation to score context integrity, quarantine suspicious documents, and provide SIEM-ready logging [13].

## 1.9 Code Embedding and Semantic Representation

### 1.9.1 The Need for Code-Specific Embeddings

General-purpose text embeddings are insufficient for capturing the complex semantics of source code, which includes both natural language (comments, identifiers) and strict syntactic structures.

Code-specific embedding models are pre-trained on massive codebases, enabling them to understand programming language semantics and the relationships between code tokens. The development of models like unixcoder, which employs a multi-head attention mechanism to capture critical code elements, facilitates tasks such as code embedding and vulnerability detection by better representing the nuanced meaning of source code.

### 1.9.2 UNIXCODER

The quality of semantic code retrieval in a RAG system depends fundamentally on the embedding model that transforms source code into vector representations. General-purpose text embedding models are insufficient for capturing the dual nature of source code, which encompasses both natural language elements (comments, identifiers) and rigid syntactic structures. For this project, UniXcoder was selected as the embedding model due to its code-specific pre-training, its integration of structural information, and its demonstrated effectiveness in vulnerability detection tasks.

UniXcoder, proposed by Guo et al. [14].At Microsoft Research in collaboration with Sun Yat-sen University, is a unified cross-modal pre-trained model for programming language that addresses a fundamental architectural limitation in earlier code pre-trained models.

Prior work fell into two categories: encoder-only models (e.g., CodeBERT) optimized for code understanding tasks but requiring an additional randomly initialized decoder for generation, and decoder-only models (e.g., CodeGPT) optimized for auto-regressive generation but sub-optimal for bidirectional understanding. Encoder-decoder unified models (e.g., CodeT5) attempted to bridge this gap but remained sub-optimal for auto-regressive tasks like code completion, which require decoder-only inference for efficiency.

UniXcoder resolves this architectural tension through two key innovations:

- Mask Attention Matrices with Prefix Adapters**:** Rather than using separate encoder and decoder architectures, UniXcoder is based on a multi-layer Transformer and utilizes mask attention matrices with prefix adapters to control the behavior of the model. This mechanism enables a single model to operate in encoder-only mode (for understanding tasks such as code search and clone detection), decoder-only mode (for generation tasks such as code completion and summarization), or encoder-decoder mode—all within the same parameter set. Compared with traditional encoder-decoder architectures, UniXcoder can be better applied to auto-regressive tasks like code completion widely used in IDEs, since the task requires a decoder-only manner for efficient inference in practice.
- Cross-Modal Content Integration**:** UniXcoder leverages two forms of auxiliary information to enhance code representation: code comments and Abstract Syntax Trees (ASTs). User-written comments provide crucial semantic information about source code (e.g., "sort a given list"), while ASTs contain rich syntax information including statement types and nested relationships.

Unlike previous pre-trained models that implicitly learn AST structure through auxiliary tasks (e.g., edge prediction), UniXcoder introduces a one-to-one mapping function that transforms an AST—a tree structure—into a sequence that retains all structural information from the original tree. This transformation enables the AST to be encoded in parallel with source code and comments as input to the Transformer, without requiring additional pre-training objectives to learn tree structure.

UniXcoder was evaluated on five code-related tasks over nine publicly available datasets: clone detection (POJ-104, BigCloneBench), code search (CSN, AdvTest, CosQA), code summarization, code generation, and zero-shot code-to-code search. For zero-shot code-to-code search, the authors constructed a dedicated dataset where the model must retrieve semantically equivalent code fragments without any task-specific fine-tuning—a scenario that directly mirrors the RAG retrieval use case in this project.

Compared with encoder-only models (Roberta, CodeBERT, GraphCodeBERT) and encoder-decoder models (PLBART, CodeT5), UniXcoder achieved state-of-the-art performance on most tasks. Further analysis revealed that both comment and AST contribute to enhancing UniXcoder's ability to capture code semantics effectively [14].

UniXcoder has been specifically validated in vulnerability detection contexts through multiple independent studies:

- Charugin et al. [15]. Conducted an experimental analysis comparing CodeBERT, GraphCodeBERT, and UniXcoder in vulnerability classification tasks using code changes extracted from CVE-associated commits. Their methodology evaluated both effectiveness (via Accuracy and F1-score) and robustness, providing a direct three-way comparison of the most prominent code pre-trained models for security applications.
- Qi et al. [16]. Proposed a semantic-preserving data augmentation technique for vulnerability detection and evaluated it on multiple code pre-trained models including CodeBERT, GraphCodeBERT, and UniXcoder. With their augmentation approach, fine-tuning on these models achieved up to a 10.1% increase in accuracy and a 23.6% increase in F1 score in the vulnerability detection task, establishing UniXcoder as an effective backbone for security-focused code analysis.
- A study on token merging for code LLMs evaluated six models CodeBERT, GraphCodeBERT, UniXcoder, CodeT5, CodeT5+ (220M), and CodeT5+ (770M) across vulnerability detection, code classification, and code translation tasks, confirming UniXcoder's suitability for production-level code analysis [17].

Critically, UniXcoder serves as a drop-in replacement for CodeBERT with no additional hardware requirements, while producing more discriminative embeddings through its contrastive learning objective. It was designed for code understanding, generation, and search tasks, and its multi-task pre-training enables it to understand code semantics rather than merely code syntax.

### 1.9.3 Contrastive Learning for Improved Embeddings

Distinguishing vulnerable from non-vulnerable code is challenging due to high inter-class similarity, especially when a patch introduces only minor changes.

Contrastive learning techniques have been employed to improve embedding separation. Mohan [18] proposed Cluster-Enhanced Supervised Contrastive Loss (CESCL), an extension of supervised contrastive learning with a distance-based regularization term that tightens intra-class clustering while maintaining inter-class separation.

When evaluated on CodeBERT and GraphCodeBERT, CESCL improved the F1 score by 1.76% on CodeBERT and 4.1% on GraphCodeBERT, demonstrating its effectiveness in code vulnerability detection and broader applicability to high-similarity classification tasks.

### 1.9.4 Abstract Syntax Tree-Augmented Representations

Beyond token-based models, incorporating program structure via Abstract Syntax Trees (ASTs) and Program Dependency Graphs (PDGs) significantly enhances representation quality.

An augmented program dependency graph approach that extends the traditional PDG to capture richer semantic and structural information, combined with optimized CodeBERT, has been shown to improve detection accuracy and F1 score on synthetic and real-world datasets by an average of up to 8.34% and 29.71%, respectively [19].

Similarly, the VDPPM method combines vector embeddings from Doc2vec and SimCodeBERT to construct high-quality code representations by leveraging path representations derived from the code's structure, further validating the importance of structural information.

HYDRA, a hybrid architecture, integrates static vulnerability rules, GraphCodeBERT embeddings, and a Variational Autoencoder (VAE) to predict latent zero-day vulnerabilities, showcasing a multi-faceted approach to semantic representation [20].

## 1.10 Retrieval Mechanisms and Vector Databases

The retrieval mechanism at the heart of a RAG system fundamentally determines whether relevant security knowledge reaches the LLM for analysis. Two distinct paradigms exist for this retrieval: lexical search and semantic search. Understanding their respective strengths, weaknesses, and the necessity of combining them is essential for designing an effective code vulnerability auditor.

### 1.10.1 Lexical Search

Lexical search refers to exact or approximate keyword matching against a text corpus. The most widely adopted algorithm is BM25 (Best Match 25), a probabilistic ranking function that scores documents based on term frequency, inverse document frequency, and document length normalization. BM25 counts exact term matches, weights them by rarity (TF-IDF), and normalizes for document length.

In traditional code auditing workflows, developers typically use grep, Ctrl+F, or regular expression-based search to locate specific patterns in source code. These tools operate on exact character-level matching and have been the standard approach for decades. Claude Code and Gemini CLI, for instance, deliberately chose to use grep-based search rather than semantic retrieval, a decision that split the developer community.

The fundamental strength of lexical search lies in its precision for exact identifiers. A rare term such as a CVE identifier (e.g., "CVE-2024-3094") appears in exactly one or very few documents in a security knowledge base. BM25 ranks that document first, every time, with no ambiguity. This property is critically important for security auditing, where developers or auditors may need to retrieve information by exact vulnerability identifiers.

Despite its precision, lexical search suffers from three fundamental limitations when applied to code vulnerability detection:

- Token Inflation: Every grep result dump floods the LLM with substantial amounts of irrelevant code, driving up token costs and increasing dramatically as the repository or knowledge base grows. The LLM must then parse through hundreds or thousands of lines to locate the genuinely relevant fragments.
- Zero Contextual Understanding: Grep matches literal strings only. It captures neither semantic meaning nor structural relationships between code elements. Searching for

"create user" cannot locate a function named addUser(), nor can it identify code that performs user creation through a different syntactic pattern. This blindness to semantics is particularly damaging for vulnerability detection, where the same flaw can manifest in structurally diverse ways across different codebases, libraries, and programming languages.

- Scalability Degradation: Lexical search re-scans the entire corpus on every query, which causes search speed to slow as the knowledge base grows larger. For a security knowledge base that must be continuously updated with newly disclosed CVEs—each potentially requiring unique query patterns—this lack of indexing becomes a significant practical bottleneck.

### 1.10.2 Semantic Search

Semantic search addresses the limitations of lexical matching by encoding both queries and documents into high-dimensional vector representations (embeddings) stored in a vector database. Search is no longer about matching strings but about computing cosine similarity between the query vector and every document vector in the index.

The process follows a two-phase pipeline. In the offline indexing phase, a pre-trained embedding model encodes every code fragment or security document into a high-dimensional vector, building a global vector index. During online querying, the user's query (which may be natural language or a code snippet) is similarly vectorized, and cosine similarity is computed against all vectors in the index. The top-k most similar fragments are returned, enabling semantic-level precision matching.

Embedding models collapse text into a fixed-dimensional vector (typically 768 or more dimensions). Tokens with similar meanings end up near each other in this vector space. That is the entire superpower—and the entire failure mode. Embeddings are not an upgrade to lexical search; they are a different tool. Using one without the other inevitably leads to retrieval failures in production.

Five distinct categories of queries reliably defeat vector-only retrieval, and several of these are directly relevant to security auditing workflows:

- Rare Identifiers: Unique strings such as CVE identifiers (e.g., CVE-2024-3094), specific error codes, or one-off vulnerability names fall squarely into this category. The embedding model has not encountered these exact tokens during pre-training. It decomposes them into sub-word fragments, averages their vector representations, and lands in a region of embedding space populated by similarly shaped identifier strings—without preserving the distinct value that makes the identifier meaningful.
- Acronyms and Abbreviations: A query for "RCE in JPA" may retrieve documents about Java ORM vulnerabilities broadly but fail to surface the particular CVE report about remote code execution in JPA that a keyword-based index would rank first. The

abbreviation collapses multiple words into a single opaque token whose embedding does not carry the full semantics its expansion would convey.

- Numeric Codes and Version Strings: Version-specific queries—"Postgres 15.4," "Python 3.11.7," or a specific CVSS score threshold—fall below the resolution of the embedding model. Minor numeric variations produce nearly identical vectors, so the retrieval returns adjacent versions rather than the exact one the user intended.
- Proper Nouns and Function Names: When a user searches for a specific function name or class identifier, vector retrieval often returns semantically related but distinct entities. The search "sanitizeInput" might retrieve documents referencing "cleanData" or "validateRequest" while missing the exact function the developer needs to examine, because the embedding model prioritizes conceptual similarity over identity matching.
- Negation and Exact Phrase Boundaries: Vector embeddings do not reliably encode negation ("not SQL injection") or phrase boundaries ("'open redirect' vs. 'open a redirect'"). As a result, semantic retrieval can return results that contradict the intended constraints or fail to respect the precise combination of terms the query specifies.

### 1.10.3 Hybrid Search Strategies

The correct architectural pattern is hybrid search: running two retrievers in parallel BM25 for exact matching and dense vectors for semantic similarity and fusing their ranked result lists into a single unified ranking. HubScan supports hybrid search, including vector similarity, hybrid search, and lexical matching with reranking capabilities [12].

### 1.10.4 Vector Database Selection

In a RAG system, the vector database serves as the retrieval engine: it stores vectorized representations of all knowledge base documents, receives a vectorized user query, performs nearest-neighbor search to identify the k most semantically similar documents, and returns the associated text fragments (and metadata) to serve as context for the LLM. The key capabilities required are efficient similarity search (via specialized algorithms such as HNSW, IVF-PQ), high-dimensional indexing, metadata storage and filtering, and scalability to handle growing knowledge bases.

### 1.10.5 ChromaDB

ChromaDB is an open-source embedding vector database specifically designed for AI applications, particularly LLM-powered RAG pipelines. It has gained significant adoption as a prototyping and development tool due to its simplicity and deep integration with the Python ecosystem.

Core features of ChromaDB:

- Lightweight and Embedded Architecture: ChromaDB's core is a Python library that can be directly integrated into application code without requiring a standalone server

deployment (though client-server mode is also supported). It installs with a single pip install chromadb command and can run embedded within the same Python process. This design enables developers to build and iterate rapidly, embedding ChromaDB into local development environments, evaluation pipelines, or applications without external infrastructure or cloud dependency.

- HNSW Indexing: ChromaDB uses the Hierarchical Navigable Small World (HNSW) algorithm for efficient approximate nearest neighbor (ANN) similarity search. HNSW constructs a multi-layered graph structure where each layer provides increasingly coarse navigation, enabling logarithmic-time search complexity for high-dimensional vectors.
- Metadata Filtering: ChromaDB supports storing structured metadata (e.g., CWE ID, programming language, severity level) alongside each vector embedding, and enables filtering search results based on these metadata fields. This is critically important for a security auditor: retrieval queries can be constrained to specific CWE categories or programming languages, ensuring that the retrieved context is not only semantically similar but also categorically appropriate.
- Persistent Storage: ChromaDB provides persistent storage through configurable backends, ensuring that the knowledge base survives process restarts and can be incrementally updated without full re-indexing.
- Simple Python API: ChromaDB offers a deliberately minimal API surface. Installing it as a Python package, embedding documents, and querying requires only a few lines of code, making it the fastest path from zero to a working prototype. For MVPs and evaluation systems, nothing is faster to set up.

### 1.10.6 Metadata Filtering

Enhancing retrieval precision requires more than just vector similarity; it necessitates filtering based on structured metadata. For a vulnerability detection system, this means that retrieval queries can be constrained by fields like `cwe_id`, `language`, or `severity`. This ensures that the retrieved context is not only semantically similar but also contextually appropriate, reducing noise and improving the LLM's ability to generate accurate and relevant analyses. The ability to perform domain-aware detection, as implemented in HubScan's domain-scoped scanning, recovers 100% of targeted attacks that evade global detection, illustrating the power of context-specific filtering [12].

## 1.11 LLM Integration & Prompting Strategies

### 1.11.1 Chain-of-Thought (CoT) Prompting

Chain-of-Thought (CoT) prompting is a technique that guides LLMs through a step-by-step “reasoning” process, making their analysis more transparent and robust. In the context of vulnerability assessment, CoT helps the model move beyond shallow pattern matching to a deeper, more contextual understanding of the code's security implications.

This approach is critical for improving decision-making quality in tasks that require nuanced judgment, such as evaluating exploitability and impact [7]. CoT is among the most effective techniques in prompt engineering, alongside in-context learning and RAG [21].

### 1.11.2 Integrating RAG with CoT

The combination of RAG and CoT represents a significant synergy. The ReVul-CoT framework demonstrates this by dynamically retrieving contextually relevant information from a local knowledge base and using CoT to guide the LLM through a reasoned assessment of the vulnerability.

This integrated approach significantly enhances LLM-based software vulnerability assessment, outperforming state-of-the-art baselines by 16.50%-42.26% in terms of Matthews Correlation Coefficient (MCC), and by 10.43%, 15.86%, and 16.50% in Accuracy, F1-score, and MCC, respectively, against the best baseline. Ablation studies within the ReVul-CoT work further validate the individual contributions of dynamic retrieval, knowledge integration, and CoT-based reasoning, confirming that the whole is greater than the sum of its parts [7].

### 1.11.3 Prompt Engineering for Code Auditing

Effective prompt engineering is essential for guiding LLMs to perform security code inspection. A study on using task-specific guidelines for secure Python code generation compared RAG with Recursive Criticism and Improvement (RCI).

The research found that both RAG and RCI deliver comparable performance in terms of code security, but RAG consumes considerably less time and fewer tokens. Furthermore, combining RAG with RCI further reduces the amount of insecure code generated, highlighting the benefit of adding relevant guidelines to improve LLM-generated code security [22].

The SOSecure project leverages collective security expertise from Stack Overflow discussions to improve the security of LLM-generated code via RAG, demonstrating another practical application of prompt-centric security knowledge [23].

### 1.11.4 Security Vulnerabilities in RAG Prompting

RAG systems themselves are susceptible to novel prompt-based attacks. The PR-Attack (Coordinated Prompt-RAG Attack) is an optimization-driven attack that poisons the knowledge database while embedding a backdoor trigger within the prompt, which when activated, can manipulate the LLM's output [24].

Similarly, the BiasRAG framework exposes fairness vulnerabilities in RAG through a two-phase backdoor attack during the pre-training phase, emphasizing the need to reassess the security of shared instructional prompts. These findings highlight that security must be considered across the entire pipeline, from knowledge base construction to prompt design [25].

## 1.12 Multi-Agent & Alternative Approaches

### 1.12.1 Agent-Based Workflows for Auditing

An alternative to monolithic LLM calls is the use of autonomous agents that can explore codebases and perform multi-step “reasoning”. RepoAudit, an autonomous LLM-agent for repository-level code auditing, exemplifies this by exploring feasible program paths and validating candidate bug reports. It does so without integrating external vulnerability knowledge (like CVE/CWE) through retrieval, instead relying on the LLM's internal “reasoning” and repository-internal retrieval of program facts [4].

### 1.12.2 Multi-Agent Systems for Vulnerability Detection

Multi-agent systems decompose the complex task of vulnerability detection into specialized, collaborative roles. Argus (Agentic and Retrieval-Augmented Guarding System) is a multi-agent framework that re-orchestrates the Static Application Security Testing (SAST) workflow to be LLM-centered, integrating techniques like RAG and ReAct to minimize hallucinations and enhance “reasoning” Argus has successfully identified critical zero-day vulnerabilities with CVE assignments [26].

Similarly, MulVul integrates retrieval augmentation with cross-model prompt evolution in a multi-agent setup for code vulnerability detection. LAMPS, another multi-agent system, employs four role-specific agents for package retrieval, file extraction, classification, and verdict aggregation to detect malicious PyPI packages, demonstrating the versatility of this architectural pattern [2].

### 1.12.3 Fine-Tuning for Vulnerability Detection

Supervised Fine-Tuning (SFT) is a common alternative that adapts a pre-trained LLM to a specific security task by training it on a labeled dataset.

SecureReviewer uses "secure-aware fine-tuning" to enhance LLM behavior for secure code review and introduces SecureBLEU, a metric for evaluating the effectiveness of security review comments [27].

While SFT can improve consistency for the target task, it introduces a strong dependence on dataset coverage and labeling quality and can be costly to update as vulnerability patterns evolve. AutoPatch, a multi-agent framework for patching real-world CVEs, highlights that frequent fine-tuning with new CVE sets is costly, and that its RAG-based approach is over 50x more cost-efficient than traditional fine-tuning approaches [6].

## 1.13 Evaluation Methodologies & Benchmarks

### 1.13.1 Evaluation Metrics for Vulnerability Detection

Evaluating vulnerability detection systems requires a nuanced approach that goes beyond simple accuracy. The use of Precision, Recall, and F1-Score is standard, but the choice of metric weighting can reflect different stakeholder priorities.

For instance, RealVuln, a benchmark for real-world code, uses the F3 score (beta=3, which weights recall nine times more than precision) to prioritize finding as many vulnerabilities as possible, a common concern for security teams [28].

The SecLens-R framework further refines this by introducing a multi-stakeholder evaluation with 35 shared dimensions and five role-specific weighting profiles (CISO, Chief AI Officer, Security Researcher, Head of Engineering, and AI-as-Actor), revealing that the same model's "Decision Score" can vary by as much as 31 points depending on the stakeholder perspective [29].

### 1.13.2 Dedicated Vulnerability Benchmarks

Several modern benchmarks have been developed to address the limitations of older, function-level datasets. RealVuln compares 15 scanners on 26 intentionally vulnerable Python repositories with 796 hand-labeled entries, establishing a clear three-tier ranking of Security-Specialized scanners > General-Purpose LLMs > Rule-Based SAST tools [28].

JITVUL is a Just-In-Time vulnerability detection benchmark built from 879 CVEs spanning 91 vulnerability types, linking each function to its vulnerability-introducing and fixing commits for a more realistic evaluation of detection capabilities [30].

SecLLMHolmes is a generalized, fully automated, and scalable framework introduced by researchers at IBM, Boston University, and UNSW for evaluating LLM performance in vulnerability detection [31].

ZeroDayBench focuses on evaluating LLM agents' ability to discover and fix previously unseen zero-day vulnerabilities in open-source codebases [32].

### 1.13.3 Challenges in Real-World Evaluation

Evaluating vulnerability detection systems on real-world code presents significant challenges. Many existing benchmarks focus on isolated, single-vulnerability samples, which fail to reflect the complexity of real-world software where multiple interacting vulnerabilities often coexist within large files [33].

Research on multi-vulnerability detection reveals that LLM performance degrades sharply as vulnerability density increases, with performance dropping by up to 40% in high-density settings

and models exhibiting severe "under-counting" in complex Python files where recall falls below 0.30 [33].

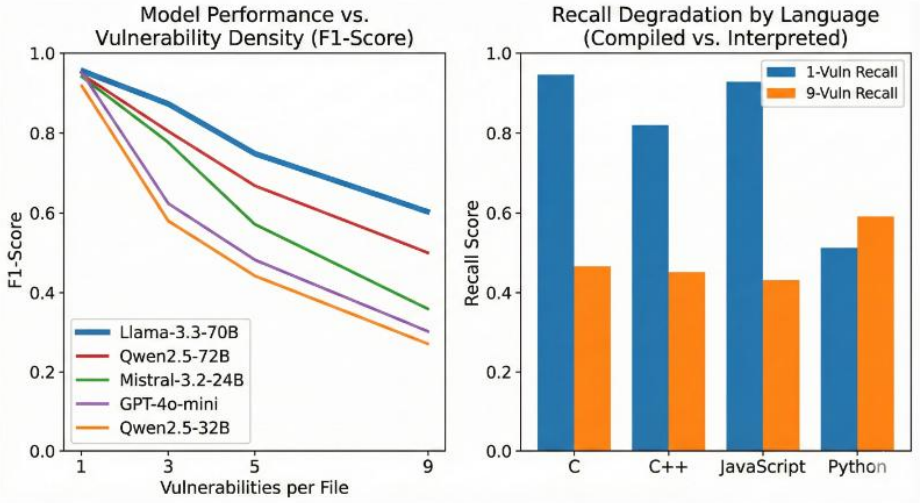


**Figure 3: Evaluation of LLMs on Code Vulnerability Detection Across Varying Density and Language Types** *[33]*.

Limitation worth considering is the data is synthetically generated which means the vulnerabilities are injected into existing code rather than occurring "in the wild." This may create artifacts (e.g., sudden style changes) that models might latch onto [33].

Furthermore, real-world detection requires inter-procedural analysis, as vulnerabilities often emerge through multi-hop function calls rather than isolated functions [30].

## 1.14 Challenges in Vulnerability Dataset Quality

### 1.14.1 Label Inaccuracy and Data Duplication

A primary obstacle in the development of automated security auditors is the low quality of existing training data. Empirical analysis of seven major vulnerability datasets reveals that widely used resources suffer from label inaccuracy rates ranging from 20% to 71%. Furthermore, data duplication is a pervasive issue; rigorous deduplication processes have shown that up to 66.94% of vulnerable-fixed pairs in common datasets are redundant or "self-identical," meaning the vulnerable code is identical to its fixed version. This lack of a clear "learning signal" in self-identical pairs prevents models from understanding the precise semantic changes required to mitigate a flaw. For example, the BigVul dataset, frequently used in research, was found to have a validity rate of only 25%, with 94.4% of its pairs providing no meaningful distinction between vulnerable and secure states [34].

### 1.14.2 Generalization Gap

In the context of LLM-based vulnerability detection, a significant "generalization gap" exists, defined as the performance difference between In-Distribution (ID) testing (using the same dataset as training) and Out-of-Distribution (OOD) testing (using independent, unseen real-world data). Research indicates that ID performance is a poor predictor of real-world effectiveness. Models often achieve misleadingly high ID scores by memorizing dataset-specific artifacts and spurious correlations rather than learning genuine vulnerability patterns. A stark example of this is the Qwen2.5-Coder model, which achieved a high 0.703 ID accuracy on BigVul but collapsed to a 0.493 OOD accuracy on real-world samples—performance essentially no better than random guessing. This underscores the necessity of retrieval-augmented approaches that ground model reasoning in verified, high-quality knowledge bases rather than relying solely on potentially overfitted parametric memory [34].

### 1.14.3 The Pitfall of “Vulnerability-Like” Code Recognition

A critical theoretical failure mode in many detection models is the "vulnerability-like" code detection pitfall. This occurs when models are trained on datasets that pair vulnerable code with general benign code from the same repository rather than their corresponding fixed versions. While these models achieve high accuracy (often >0.96) at identifying code that "looks" like a vulnerability, they fail to distinguish between a flaw and its specific fix [34]. This suggests the model has not learned the precise, semantic nature of the vulnerability itself. To be effective, an auditor must be able to recognize the subtle textual differences—such as relocated method invocations or modified conditional checks—that separate a risk from a remediation [5].

### 1.14.4 Data Sparsity and Class Imbalance

The vulnerability landscape is characterized by severe class imbalance, which creates a theoretical bottleneck for detecting rare but dangerous weaknesses. In consolidated vulnerability databases, the top 5–10 CWEs typically comprise 55% to 80% of all available samples. The frequency ratio between the most common type (e.g., CWE-20: Improper Input Validation) and the least common (e.g., CWE-798: Use of Hard-coded Credentials) can be as high as 130:1. This data sparsity directly impacts model performance; models trained on imbalanced data perform poorly on "long-tail" CWE categories, regardless of their architectural sophistication. This justifies the use of synthetic data generation (RVG) to augment rare categories, which has been shown to improve detection accuracy on real-world data by 5.8% to 13.1% [34].

## 1.15 Practical Implementation Considerations

### 1.15.1 Lightweight vs. Heavyweight Architectures

The comparison by Saju et al. [1]. Supports our proposed system which emphasizes a lightweight and practical design, as the RAG approach achieved the highest performance without the orchestration complexity of a multi-agent system or the retraining cost of fine-tuning.

The token efficiency of RAG, as demonstrated by the comparison with RCI, further underscores its suitability for a practical, resource-conscious auditor [22].

### 1.15.2 Explainability and Traceability

A key advantage of RAG-based systems is their inherent explainability. Because the LLM's analysis is grounded in specific, retrieved documents, the system can provide source-linked explanations for its findings.

Vul-RAG's generated knowledge served as high-quality explanations that improved manual detection accuracy from 60% to 77%, demonstrating the practical value of this traceability [5].

### 1.15.3 Continuous Knowledge Updates

One of the most significant practical benefits of RAG is that it enables continuous updates to the system's knowledge and delivers precise, current responses by extracting relevant items from a vector database without requiring expensive and time-consuming model retraining.

This is critical for maintaining relevance against the rapidly evolving landscape of newly disclosed CVEs, a challenge that frameworks like AutoPatch address by using RAG with a structured database of recently disclosed vulnerabilities [6].

### 1.15.4 Security of the RAG Pipeline

Building a security tool requires securing the tool itself. The literature highlights several critical vulnerabilities in RAG pipelines, including knowledge base poisoning and prompt injection attacks.

A comprehensive review of threats, defenses, and benchmarks for RAG systems analyzes the underlying vulnerability mechanisms and categorizes threat vectors such as data poisoning, adversarial attacks, and membership inference [35].

A new class of "Retrieval Pivot Attacks" has been identified in hybrid RAG pipelines that combine vector similarity search with knowledge graph expansion, where attackers can amplify leakage from initial vector retrieved seeds (chunks) to access the expanded graph, highlighting the need for careful security consideration in these advanced architectures [36].

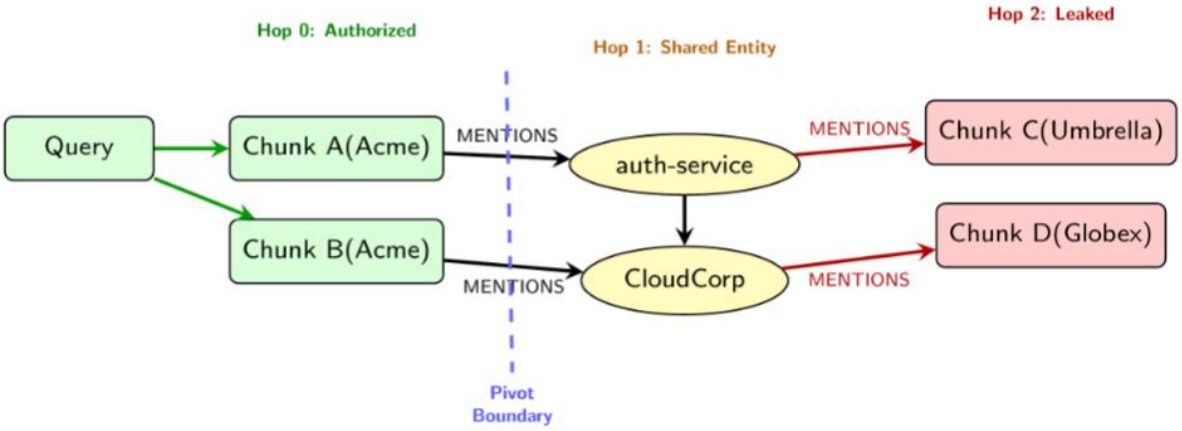


**Figure 4: The retrieval pivot attack surface** *[36]*.

Vector retrieval returns authorized seed chunks (hop 0, green). BFS expansion traverses MENTIONS edges to shared entity nodes (hop 1, yellow), then onward to unauthorized chunks from other tenants (hop, red). The pivot boundary (dashed) is where authorization must be re-checked. D1 filters at this boundary, eliminating all leakage [36].

Tools like RAGShield, which sits between retrieval and generation to score context integrity and quarantine suspicious documents, represent a new class of security controls for RAG applications [13].

## 1.16 Conclusion

This chapter presented the theoretical foundation of the proposed RAG-based security auditor. It discussed the role of Large Language Models in source code analysis, the limitations of relying only on internal model knowledge, and the importance of grounding security analysis in external knowledge sources such as CVE, CWE, and OWASP. The chapter also explained how Retrieval-Augmented Generation can improve vulnerability detection by combining semantic retrieval with LLM-based reasoning.

The reviewed concepts show that RAG is suitable for this project because it allows the system to retrieve relevant vulnerability knowledge at analysis time without requiring continuous fine-tuning. In addition, the discussion of embeddings, vector databases, knowledge base construction, and prompting provides the technical basis for building a practical and explainable security auditing pipeline.

In summary, the theoretical background supports the main direction of this project: combining LLM reasoning with structured cybersecurity knowledge to produce more reliable and explainable code security audit reports. The following chapter builds on this foundation by describing the preparation of the working environment and the tools used to implement the

proposed system. Based on this foundation, the next chapters translate these concepts into a practical system environment and implementation pipeline, using SecureCode v2.0 as the knowledge source, UniXcoder for code representation, ChromaDB for vector storage, LangChain for orchestration, and an OpenAI GPT model for generating structured security audit reports.

# Chapter 2: Literature Review

## 2.1 Introduction

This chapter reviews prior research on automated software vulnerability detection and AI-assisted security auditing, with particular focus on recent Large Language Model (LLM)–based approaches. Across the surveyed works, a consistent motivation emerges: security flaws can be subtle and context-dependent, while the scale and rapid evolution of modern software make exhaustive manual auditing difficult. As a result, many studies explore how LLMs can support vulnerability identification, explanation generation, and even remediation. However, empirical evidence indicates that LLM-based security judgments can be sensitive to prompt design, training data scope, and the availability of external evidence, which is especially problematic in security-critical settings where incorrect conclusions can create false assurance or unnecessary triage effort [1]. The following review is organized thematically to compare how prior work operationalizes LLMs for security: (i) prompt-only inspection, (ii) fine-tuning and supervised adaptation, (iii) agent-based and multi-agent workflows, (iv) retrieval-augmented and knowledge-grounded systems, (v) knowledge-base construction and vulnerability representation, and (vi) evaluation practices and benchmarks. Collectively, these themes motivate the design choices behind the proposed RAG-based Security Auditor, which prioritizes evidence-grounded reasoning and structured reporting.

## .2 LLM-only prompting for security code inspection

A foundational baseline in the contemporary literature is prompt-based security inspection, where a general-purpose GPT-style model is given source code and asked to identify vulnerabilities and propose fixes. “A New Approach to Web Application Security: Utilizing GPT Language Models for Source Code Inspection” exemplifies this direction by framing the LLM as an end-to-end analyzer driven primarily by natural-language instructions [9]. The key advantage of prompt-only inspection is low engineering overhead: it requires no dedicated training pipeline and can be applied quickly across different code artifacts. Yet the same simplicity exposes important limitations. Because the model’s decision is primarily conditioned on the prompt and its internal knowledge, reliability can vary substantially across prompt formulations and across vulnerability types, and the output may lack stable provenance when the model does not explicitly ground claims in external vulnerability evidence [1], [9]. This baseline therefore motivates subsequent work that strengthens LLM-based auditing through either task-specific adaptation (fine-tuning), process modeling (agents), or external knowledge grounding (retrieval).

## 2.3 Fine-tuning and supervised adaptation for secure code review

A second line of research attempts to improve LLM performance by adapting models to security tasks through supervised fine-tuning (SFT) or security-aware training. SecureReviewer explicitly proposes “secure-aware fine-tuning” to enhance LLM behavior in secure code review scenarios, reflecting the view that generic pretraining alone does not consistently yield security-correct judgments across diverse code contexts [8]. RealVul similarly investigates whether LLMs can

detect vulnerabilities in web applications, representing a task- and dataset-driven approach to improving detection through targeted training and evaluation [3]. Compared with prompt-only inspection, fine-tuning can improve consistency because a model is optimized for the target task; however, the literature also highlights that fine-tuning introduces stronger dependence on dataset coverage and labeling quality, and it can be costly to update as vulnerability patterns and best practices evolve [1], [8]. This trade-off is important for the project because a practical auditor must remain maintainable over time, motivating solutions that reduce the need for frequent retraining while still improving reliability.

## 2.4 Agent-based and multi-agent auditing workflows

Because real security auditing is iterative—often requiring context gathering, hypothesis testing, and validation—several works model vulnerability analysis as an explicit workflow rather than a single prediction. RepoAudit exemplifies this process-oriented view by proposing an autonomous LLM agent for repository-level code auditing, shifting attention from isolated functions toward codebase exploration and multi-step reasoning across files and components [4]. This approach is motivated by the observation that vulnerabilities frequently depend on broader program context (e.g., how inputs propagate or how components interact), which single-shot prompting may not capture reliably. Multi-agent research extends this workflow perspective by decomposing the auditing task into specialized roles or cooperative behaviors. MulVul proposes a retrieval-augmented multi-agent system, indicating that distributing responsibilities across agents can help address heterogeneity in vulnerability patterns and can enable more structured exploration and refinement during detection [2]. AutoPatch further demonstrates a multi-agent architecture in a closely related setting—patching real-world CVEs generated by outdated LLMs—suggesting that agent coordination can support complex, multi-step security workflows beyond detection alone [6]. At the same time, agent-based systems introduce practical concerns: orchestration complexity, higher computational overhead, and difficulty scaling workflows to large repositories without careful control of context and verification steps [1], [2], [4]. These limitations motivate the project to seek a lighter-weight auditing pipeline that still supports strong evidence grounding and structured reporting.

## 2.5 Retrieval-augmented and knowledge-grounded vulnerability analysis

A major contemporary trend is to ground LLM reasoning in external security knowledge retrieved at inference time. The empirical evaluation in [1] frames Retrieval-Augmented Generation (RAG) as a primary family of approaches alongside SFT and dual-agent systems, reflecting the growing consensus that retrieval can mitigate limitations associated with static parametric knowledge and prompt sensitivity [1]. Vul-RAG is particularly relevant to this project because it proposes a knowledge-level RAG framework for vulnerability detection, emphasizing that retrieving distilled vulnerability knowledge (rather than only code-near snippets) can guide an LLM to reason about vulnerability behavior, causes, and fixes more effectively [5]. In parallel, ReVul-CoT proposes combining retrieval augmentation with structured reasoning

prompts (Chain-of-Thought), indicating that retrieval alone may be insufficient unless coupled with a disciplined reasoning process that uses retrieved evidence to justify conclusions [7]. RAG also appears in multi-agent settings: MulVul integrates retrieval into a multi-agent detection pipeline, and AutoPatch uses a multi-agent framework to patch vulnerabilities that arise due to outdated LLM knowledge, both reinforcing the view that retrieval is increasingly treated as a practical mechanism for keeping security reasoning aligned with evolving vulnerability information [2], [6]. For the project, these works collectively motivate an auditor architecture centered on evidence retrieval + reasoning, aiming to improve reliability and explainability without relying solely on extensive fine-tuning.

## 2.6 Knowledge-Base construction and vulnerability representation

Across retrieval-centered systems, multiple studies emphasize that "what" is retrieved matters as much as "how" it is retrieved. Vul-RAG motivates multi-dimensional vulnerability knowledge—including functional semantics, root causes, and fixing solutions—suggesting that abstract, generalizable representations can guide auditing more effectively than narrow lexical similarity [5]. ReVul-CoT similarly foregrounds building a local knowledge base that fuses multiple vulnerability information sources, aligning with the argument that richer, structured knowledge can improve both decision quality and interpretability when used as retrieved context during assessment [7]. AutoPatch reinforces the operational importance of maintainable vulnerability knowledge by explicitly targeting real-world CVEs and highlighting the practical risk of outdated model knowledge, which retrieval-based designs attempt to address without continuous retraining [6]. These perspectives directly inform the project's goal of constructing a structured security knowledge base (mapped to vulnerability identifiers and weaknesses) to support traceable findings and actionable remediation guidance.

## 2.7 Evaluation and benchmarking practices

Credible security tooling depends on evaluation protocols that reflect real-world difficulty and operational constraints. The empirical comparison in [1] highlights the need to evaluate multiple methodological families (RAG, SFT, and dual-agent or agent-assisted settings) under consistent criteria, making explicit that reported performance and practical reliability can vary substantially across approach classes [1]. RealVul and SecureReviewer represent task-focused studies that evaluate LLM performance under curated settings for vulnerability detection and secure review, providing evidence that targeted adaptation can improve outcomes but may also bind performance to dataset scope and update frequency [3], [8]. Vul-RAG and ReVul-CoT highlight evaluation settings where retrieval and reasoning strategies are tested as part of the overall detection/assessment pipeline, reinforcing that the "system" (retrieval + reasoning + reporting) is often the unit of evaluation rather than the LLM alone [5], [7]. For the project, these evaluation practices motivate designing an auditor that can be assessed not only on detection correctness but also on explanation quality, mapping accuracy (e.g., to weakness/vulnerability references), and the practical usefulness of remediation recommendations.

In summary, the reviewed literature proposes complementary strategies for strengthening LLM-based security auditing: prompt-based inspection as an accessible baseline [9], supervised fine-tuning for improved task alignment [3], [8], agent and multi-agent workflows for process-oriented auditing and remediation [2], [4], [6], and retrieval-augmented systems that ground reasoning in external vulnerability knowledge [1], [5], [7]. These themes correspond directly to the dimensions synthesized in the existing comparison table—namely the approach category, dataset dependence, scalability considerations, grounding mechanism, and principal limitations—providing the basis for a structured comparison of prior research directions before motivating the project approach.

**Table 1 Literature Review**

| Ref. | Scope | Retrieval target | LLM models | Evaluation | Limitation |
|---|---|---|---|---|---|
| [27] | Secure code review comment generation and security issue detection; focuses on PR/code-diff review. | RAG retrieves review templates. | CodeLlama-7B, DeepSeek-Coder-6.7B, Qwen2.5-Coder-7B, Claude-3.5-Sonnet. | Issue Detection: Models outperform all baseline metrics. Domain-specific fine-tuning was critical, yielding an absolute F1-score increase of 64.87 for CodeLlama-7B and 30.27 for Qwen2.5-Coder-7B.<br>Comment Quality: SecureReviewer(CL) leads with a 11.34 BLEU-4 score (beating DeepSeek-V3 by 5%) and a 29.31 SecureBLEU score (surpassing LlamaReviewer's 24.56).<br>Key Enhancements:<br>Fine-tuning: Boosted CodeLlama-7B's BLEU-4 by 7.65 and SecureBLEU by 16.2.<br>SA-Loss: Using a secure-aware re-weighted loss function further optimized security-critical token detection.<br>RARG: Significantly improved SecureBLEU scores by providing specialized security knowledge that general-purpose LLMs lack. | Limited contextual awareness (diff-only); dataset size constrains model scale; templates built from same dataset may limit generalization. |

| | | | | | |
|---|---|---|---|---|---|
| [4] | LLM agent audits large repositories despite context limits and hallucinations by exploring feasible program paths and validating candidate bug reports. | Repository-internal retrieval: demand-driven function/path exploration and retrieval of program facts (data-flow facts + path conditions. | Claude-3.5 Sonnet (core system); additional evaluation with DeepSeek-R1, Claude-3.7 Sonnet, and OpenAI o3-mini. | Performance:<br>Achieves a precision of 78.43% (40 true positives and 11 false positives). It successfully detected all previously reported bugs, plus 19 new historic bugs (14 of which are already fixed).<br>LLM Comparison:<br>While analyzing relevant functions, Chain-of-Thought (CoT) prompting detected only 1 true bug (single-function) and 10 true bugs (multiple-function).<br>The agent-centric LLMDFA proved highly expensive, requiring 165.23 times more prompting rounds and 123.18 times more tokens on average.<br>Ablation Study:<br>Without abstraction:<br>True positives dropped by 47.50% and false positives increased by 181.82%.<br>Without validators:<br>False positives increased by 245.45%.Without caching: Costs were 3 to 4 times higher on average, and up to 30 times higher in the worst case. | Does not integrate external vulnerability knowledge (e.g., CVE/CWE) through retrieval mechanisms such as RAG, relying mainly on the LLM's internal reasoning. |
| [1] | Controlled comparison of RAG vs SFT vs Dual-Agent for vulnerability detection. | RAG retrieves knowledge chunks (security/vulnerability context) for the prompt; primarily CWE-oriented. | Base: LLaMA 3.2-3B; Dual-Agent = detector + validator model (same base family) | RAG achieved the highest overall performance with an average F1 Score of 0.85, significantly outperforming SFT (0.80), Dual-Agent LLM (0.77), and the Base Model (0.67). | Limited CWE breadth (5). |

| | | | | | |
|---|---|---|---|---|---|
| [3] | Proposes an LLM-based framework for PHP web vulnerability detection, addressing weaknesses in PHP vulnerability sampling/processing and dataset scarcity. | None (no retrieval component; relies on fine-tuned code LLMs). | CodeT5-base, CodeT5+, CodeLlama-7B, StarCoder2-3B, and StarCoder2-7B (fine-tuned for vulnerability detection); additional evaluation with GPT-4, Llama-3-8B, and Mistral-7B. | RealVul was evaluated on CWE-79 (XSS) and CWE-89 (SQLI) vulnerabilities using synthesized training data, showing strong performance across four metrics.<br>Effectiveness: RealVul generally outperformed the baseline. The best results came from combining RealVul with CodeLlama-7b and StarCoder2-3b. Notably, while the baseline's F1 score remained below 50%, RealVul's sampling techniques significantly improved the model's ability to represent and detect vulnerability features.<br>Generalization: Despite a small proportion of vulnerability samples in the training set (mimicking real-world imbalances), RealVul's F1 score significantly surpassed the baseline. While slightly less accurate than the baseline in some areas, it handled the complexity of vulnerability detection better than the baseline's "complete function" approach.<br>Ablation Study: Normalization is typically essential, even though StarCoder2-3b saw some F1 decreases on unseen projects. The maximum F1 score difference reached 14.06%, suggesting that removing irrelevant semantics—and specifically normalizing function names—improves PHP vulnerability detection. | Heuristic rule-based vulnerability trigger detection and per-CWE fine-tuned models limit scalability and generalization. During the normalization process, they preserved user-defined function names. They fine-tuned the model specifically for each type of CWE vulnerability to augment detection capabilities, this approach incurs substantial overhead and the performance of a unified multi-classification model merits investigation. |

| | | | | | |
|---|---|---|---|---|---|
| [37] | Uses GPT models to perform static source code inspection for web/front-end code, with preprocessing and a prompt pipeline (few-shot + chain-of-thought) to detect sensitive code segments and map the code into manageable JSON structures. | None (no retrieval mechanism; GPT analyzes code directly via prompt-based pipeline). | GPT-3.5 and GPT-4 via OpenAI API. | The detection success rate has jumped from 45% to 88.76% with a tightening approach of deeming a service sensitive across the entire application if it was categorized as handling sensitive data in any component of the application. Consequently, it was assigned the highest sensitivity level observed in any affected component. | Relies on prompt-based GPT analysis without external knowledge integration and was evaluated on a limited set of Angular applications. |

The comparison in Table 1 demonstrates the rapid adoption of Large Language Models (LLMs) for software security auditing and vulnerability detection tasks. Several recent works explore different strategies to enhance the reliability of LLM-based analysis. For example, SecureReviewer [1] focuses on improving the quality of security-oriented code review comments through domain-specific fine-tuning and retrieval of curated review templates, showing that combining fine-tuning with limited retrieval grounding can significantly improve the usefulness of automated security reviews. Similarly, the empirical study in [3] systematically compares different approaches—Retrieval-Augmented Generation (RAG), supervised fine-tuning, and dual-agent architectures—for vulnerability detection, demonstrating that incorporating external vulnerability knowledge (e.g., CWE descriptions) via RAG leads to the best overall performance, achieving an F1 score of 0.85. Other works explore alternative directions. RepoAudit [2] introduces an LLM-agent capable of auditing large code repositories by exploring feasible program paths and validating candidate bug reports, showing that autonomous reasoning agents can overcome context limitations when analyzing complex codebases. In contrast, RealVul [4] investigates the use of fine-tuned code LLMs for detecting vulnerabilities in PHP web applications, addressing dataset scarcity and demonstrating that specialized training can improve vulnerability classification performance. Additionally, the approach proposed in [5] utilizes GPT models for static source code inspection through prompt-based pipelines to detect sensitive code segments in web applications.

Despite these advances, the existing literature reveals several limitations. Many approaches rely either on fine-tuned models trained on specific datasets or on prompt-based reasoning without integrating structured security knowledge sources, which can limit their generalization and security awareness. Even when retrieval is employed, as in [1] and [3], the retrieved information is often restricted to templates or limited vulnerability descriptions rather than comprehensive security standards. Furthermore, some methods focus on specific programming languages or vulnerability types, which reduces their applicability to broader security auditing scenarios.

Motivated by these limitations, this project proposes a RAG-based security auditor that integrates external vulnerability knowledge sources, such as CVE databases and OWASP Top 10 security guidelines, to support automated code auditing and compliance verification. By grounding the LLM's analysis in structured and up-to-date security knowledge, the proposed system aims to provide more reliable vulnerability detection and generate structured security reports that assist developers in identifying and mitigating potential security flaws in their code

# Chapter 3: Environment Setup

## 3.1 Introduction

This chapter describes the technical environment and component stack selected for implementing the RAG-based security auditor. The system is constructed as a modular pipeline in Python, where each component—from data ingestion to vulnerability report generation—is implemented as a discrete, replaceable stage. The selection of each component was guided by the project's design requirements: a lightweight, local-first architecture suitable for prototyping and demonstration within a graduation project timeline, while maintaining sufficient technical rigor to produce meaningful security audit results grounded in real-world vulnerability knowledge. The following sections detail the rationale, technical specifications, and role of each component: the Python programming language and LangChain orchestration framework, the SecureCode v2.0 vulnerability dataset, the UniXcoder code embedding model, the ChromaDB vector database, and the OpenAI GPT model serving as the “reasoning” engine.

## 3.2 Development Environment

The prototype was implemented in a Python-based development environment designed to support the two main phases of the proposed system: offline knowledge base construction and online vulnerability analysis. In the offline phase, the environment supports loading and processing vulnerability datasets, generating semantic code embeddings using UniXcoder, and storing the resulting vectors and metadata in ChromaDB. In the online phase, the same environment supports receiving user-submitted source code, converting it into a query embedding, retrieving semantically similar vulnerable examples, constructing a context-aware prompt, and sending the final prompt to the GPT-4o model for vulnerability analysis and report generation.

The development environment was selected to be lightweight, modular, and suitable for a prototype implementation. Instead of requiring heavy model training or GPU-based fine-tuning, the system relies on pretrained code representation models, vector similarity search, and API-based LLM reasoning. This makes the prototype practical for experimentation while still allowing the integration of real-world vulnerability knowledge, including metadata such as CVE, CWE, programming language, and OWASP categories.

### 3.2.1 Hardware Environment

The proposed prototype was developed and tested on a local development workstation. The machine used for implementation was an ASUS TUF Gaming F15 FX507ZV4/FX507ZV laptop running on a 12th Gen Intel(R) Core(TM) i7-12700H processor with a base frequency of 2.30 GHz. The device was equipped with 16 GB of RAM, of which approximately 15.6 GB was usable by the operating system. The system architecture was 64-bit, based on an x64 processor.

This hardware environment was sufficient for implementing the proposed RAG-based security auditor because the project did not require training or fine-tuning a large language model locally. The local machine was mainly used for dataset preprocessing, code embedding generation using

UniXcoder, vector database construction using ChromaDB, and execution of the Python-based pipeline. The most computationally intensive reasoning task was performed through the OpenAI GPT-4o API, which reduced the need for high-end local GPU resources.

The use of this local workstation provided a practical and reproducible environment for prototype development. It allowed the offline knowledge base construction process and the online inference pipeline to be tested in a realistic student development setting, while maintaining acceptable performance for embedding generation, similarity search, and structured security report generation.

### 3.2.2 Software Environment

The software environment was built mainly using Python, which served as the core programming language for implementing the RAG-based pipeline. Python was selected because of its strong ecosystem for machine learning, natural language processing, vector databases, and API integration. The implementation uses LangChain to organize the main pipeline components, including document preparation, retrieval, prompt construction, and interaction with the LLM. This helps keep the system modular and easier to extend in later development stages.

For semantic code representation, the prototype uses UniXcoder, specifically the microsoft/unixcoder-base model, to convert source code snippets into vector embeddings. These embeddings are stored in ChromaDB, which acts as the vector database for similarity-based retrieval. During analysis, user-submitted code is embedded using the same model and compared against the stored vulnerability embeddings to retrieve the most relevant examples. The retrieved examples and their metadata are then combined into a structured prompt and passed to GPT-4o through the OpenAI API to generate the final vulnerability analysis, CWE/OWASP mapping, risk explanation, and recommended fixes.

The main software components used in the environment include Python, LangChain, UniXcoder, ChromaDB, and the OpenAI API. Together, these tools provide the necessary foundation for implementing both the offline knowledge base construction phase and the online inference phase of the proposed security auditor.

## 3.3 Tools and Libraries

This section presents the main tools and libraries used to implement the proposed RAG-based security auditor prototype. The selected components were chosen to support the complete workflow of the system, including dataset processing, code embedding generation, vector storage, semantic retrieval, prompt construction, and LLM-based vulnerability analysis. Each tool contributes to a specific stage of the pipeline, allowing the system to be implemented in a modular and practical manner without requiring local fine-tuning of a large language model.

### 3.3.1 Python & LangChain

Python was selected as the primary programming language for this project. Python is the de facto standard for machine learning and natural language processing research, hosting the entire ecosystem of libraries required for the RAG pipeline: sentence-transformers for embedding generation, chromadb for vector storage, and langchain for pipeline orchestration. Its interpreted, dynamically typed nature enables rapid prototyping, and its dominance in the LLM application space ensures that all selected components offer first-class Python APIs and integration support.

LangChain serves as the orchestration framework that connects the individual stages of the RAG pipeline into a coherent application. LangChain is an open-source Python framework specifically designed for building applications powered by large language models. It provides a standardized set of abstractions for the core operations required in any RAG system: document loading and preprocessing, text splitting (chunking), embedding generation, vector store integration, retrieval, and prompt construction. Rather than requiring developers to implement custom integration code between each stage, LangChain offers pre-built components RecursiveCharacterTextSplitter, HuggingFaceEmbeddings, Chroma vector store wrapper, RetrievalQA chain that can be configured and composed declaratively.

The framework also provides a PromptTemplate abstraction that enables systematic construction of prompts combining retrieved security knowledge with the source code under analysis, ensuring that the LLM consistently receives well-structured auditing instructions. For this project, LangChain reduces the integration complexity of the pipeline, allowing effort to be focused on the security-specific logic, knowledge base quality, retrieval strategy, and prompt design rather than on infrastructure code. LangChain's architecture is modular: each component (retriever, embedding model, LLM) can be swapped without restructuring the pipeline, which is valuable for a graduation project where different configurations may need to be evaluated.

Extensive community and industry adoption validates LangChain's suitability. It integrates with over 50 LLM providers and more than 30 vector database backends, including ChromaDB which is used in this project. Practical tutorials demonstrate building complete RAG pipelines with LangChain, ChromaDB, and embedding models in a matter of minutes. The framework is well-documented and maintained, with active development through 2025 ensuring compatibility with current LLM APIs and vector database versions.

### 3.3.2 SecureCode v2.0 Dataset

The knowledge base that grounds the system's security analysis is constructed from SecureCode v2.0, a production-grade dataset of security-focused coding examples designed to train security-aware AI code generation models. SecureCode v2.0 was selected as the primary data source for three reasons: its comprehensive coverage of modern vulnerability categories, its grounding in

real-world security incidents, and its systematic structure that facilitates extraction of vulnerable code, metadata, and security descriptions.

Dataset Composition: SecureCode v2.0 contains 1,215 rigorously validated coding examples that have passed both structural validation and expert security review. The dataset covers 11 vulnerability categories, including the complete OWASP Top 10:2025 plus an additional AI/ML Security Threats category. Examples span 11 languages: Python, JavaScript, Java, Go, PHP, C#, TypeScript, Ruby, Rust, Kotlin, and YAML for infrastructure-as-code configurations. Each example is structured as a multi-turn conversation between a developer and an AI security assistant, modeling realistic code review interactions.

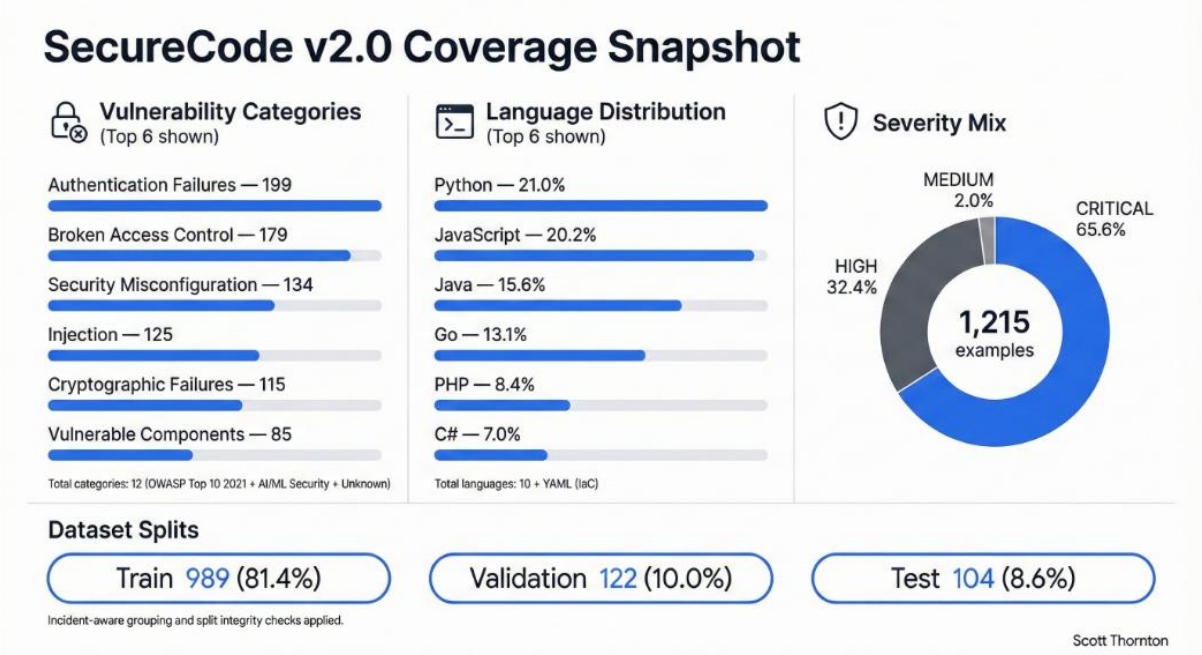


**Figure 5: Secure-Code v2.0 Coverage** *[10]*.

Real-World Incident Grounding: A distinguishing characteristic of SecureCode v2.0 is that every example is rooted in a documented security incident with associated CVE identifiers. The dataset draws from actual security events spanning 2017–2025, such as the MoveIt Transfer SQL injection that impacted over 2,100 organizations with estimated damages of $92 billion. This CVE-grounded construction ensures that the knowledge base reflects real vulnerability patterns rather than synthetic or hypothetical examples. For this project, this grounding is critical: when the system retrieves a vulnerability example matching the input code, the LLM can reference the associated real-world incident to provide context-rich, evidence-backed audit reports.

- Extraction and Dataset Preparation:
  For the purpose of building the knowledge base, the project uses a SecureCode-derived CSV dataset named vulnerable_code_combined.csv. This file contains vulnerable code

snippets together with their associated security metadata, including the example ID, CWE identifier, OWASP category, programming language, CVE reference when available, CVSS score when available, and a natural-language incident description. The implementation does not parse the original SecureCode JSONL files during the current system execution. Instead, the CSV file is treated as the prepared dataset used by the ingestion pipeline. During ingestion, the system reads this CSV file, validates that the required columns are present, removes records that do not contain vulnerable code, and converts each remaining row into a document suitable for vector storage.

- Role in the System:
  The prepared CSV dataset serves as the source material for the vector knowledge base. In the current prototype, each vulnerable_code field is treated as one complete document and embedded directly using UniXcoder. No AST-aware chunking or function-level splitting is applied at this stage. The related metadata fields, such as cwe_id, owasp_category, language, cve_id, cvss, and incident_description, are stored alongside each document in ChromaDB. During retrieval, the system retrieves semantically similar vulnerable code examples and passes their metadata to the LLM as supporting evidence. This allows the LLM to ground its analysis in known vulnerability examples and produce a structured JSON output containing the verdict, likely CWE/OWASP mapping, risk explanation, remediation guidance, and retrieval-based evidence. Metadata filtering by language or vulnerability type is not applied in the current prototype, but it is considered a future enhancement.

### 3.3.3 UniXcoder

UniXcoder is the embedding model responsible for converting source code into the numerical vector representations that power the system's semantic retrieval capability. UniXcoder is a unified cross-modal pre-trained model for programming language. It was selected over general-purpose text embedding models and alternative code models because of its architectural design for code representation, its demonstrated effectiveness in vulnerability detection tasks, and its practical deployment characteristics.

#### 3.3.3.1 Architecture and Training

UniXcoder is based on a 12-layer Transformer architecture with a hidden size of 768 dimensions and 12 attention heads, totaling 125 million parameters—identical in scale to CodeBERT-base. What distinguishes UniXcoder from its predecessors is its unified architecture: using mask attention matrices with prefix adapters, a single model can operate in encoder-only mode (for understanding tasks such as code embedding and clone detection), decoder-only mode (for generation tasks such as code completion), or encoder-decoder mode. This flexibility ensures that in encoder-only mode—the mode used for embedding generation in this project—the model produces high-quality bidirectional representations of code.

UniXcoder was pre-trained on multimodal data including source code, associated natural language comments, and Abstract Syntax Tree (AST) representations. The incorporation of AST information during pre-training is particularly significant: it enables the model to capture syntactic structure alongside semantic meaning. For vulnerability detection, many security flaws manifest as structural patterns—untrusted input flowing into a sensitive sink, missing authorization checks before privileged operations—that are best represented through combined syntactic and semantic encoding. UniXcoder's pre-training objectives include Masked Language Modeling, Cross-Modal Generation (aligning code with natural language descriptions across programming languages), and a contrastive learning objective that pulls semantically similar code fragments closer while pushing dissimilar code apart in the embedding space.

UniXcoder has been explicitly evaluated in vulnerability detection contexts –as stated earlier in the theoretical study. Experimental analysis comparing CodeBERT, GraphCodeBERT, and UniXcoder on vulnerability classification using code changes extracted from CVE-associated commits, evaluating both accuracy and robustness. Other framework leverages an ensemble of CodeBERT, GraphCodeBERT, and UniXcoder exploiting UniXcoder's cross-modal capabilities to improve vulnerability detection, highlighting the complementary strengths of different code pre-trained models. Numerous open source project uses UniXcoder embeddings within a RAG architecture to detect, explain, and remediate security vulnerabilities across multiple programming languages demonstrating the practical viability of UniXcoder in a RAG pipeline structurally similar to this project's architecture.

#### 3.3.3.2 Deployment Characteristics

UniXcoder is available as a pre-trained model on Hugging Face under the identifier microsoft/unixcoder-base. It produces 768-dimensional embeddings, which match the default dimensionality of the ChromaDB index. At 125 million parameters, UniXcoder can run efficiently on CPU hardware during inference, with embedding generation for a typical code snippet completing within tens of milliseconds. This CPU-friendly profile makes it suitable for a local-first, self-contained auditing tool that does not require dedicated GPU infrastructure. UniXcoder is released under the Apache 2.0 license, permitting unrestricted use in research and educational projects.

#### 3.3.3.3 Role in the System

UniXcoder is loaded via the sentence-transformers library and wrapped as a LangChain-compatible embedding class. During the knowledge base construction phase, every code chunk extracted from the SecureCode dataset is passed through UniXcoder to generate its 768-dimensional vector representation. During the auditing phase, the user's submitted source code is similarly embedded, and the resulting query vector is used to retrieve the most semantically similar vulnerability examples from the ChromaDB index via cosine similarity search. UniXcoder's code-specific pre-training ensures that the retrieved examples are relevant to the semantic structure of the query code, rather than being matched on superficial lexical features.

### 3.3.4 ChromaDB

ChromaDB is the vector database that stores, indexes, and retrieves the embedded vulnerability knowledge base. It is an open-source, embedding-native database specifically designed for AI applications, particularly RAG pipelines powered by large language models.

#### 3.3.4.1 Technical Capabilities

ChromaDB supports the Hierarchical Navigable Small World (HNSW) algorithm for efficient approximate nearest neighbor search, enabling retrieval of the top-k most similar vectors from the index in near-logarithmic time. At the scale of this project's knowledge base approximately 1,200 code examples generating fewer than 5,000 total vectors—ChromaDB operates well within its optimal performance envelope. Independent benchmarks indicate that at 10,000 vectors, ChromaDB returns results within 2 ms, and production reports confirm that a single VPS with 4–8 GB of RAM handles millions of embeddings comfortably in embedded mode [38].

A critical feature for this project is ChromaDB's support for structured metadata filtering. Each stored vector is accompanied by a metadata dictionary containing fields such as cwe_id, language, owasp_category, severity, and the raw code string itself. ChromaDB's where clause API enables queries to be constrained by these metadata fields. For example, when analyzing Python code, the retrieval can be restricted to Python-specific vulnerability examples, or when searching for SQL injection patterns, the search space can be narrowed to CWE-89 examples specifically. This prevents the retriever from returning semantically similar but categorically irrelevant results—such as a JavaScript XSS example when the user has submitted Python backend code.

#### 3.3.4.2 Persistent Storage and Integration

ChromaDB provides persistent storage via its configurable backend, ensuring that the constructed knowledge base survives process restarts without requiring re-indexing. It integrates directly with LangChain through the langchain_chroma package, enabling the vector store to be used as a retriever within LangChain's RAG chain architecture. The database installs with a single pip install chromadb command and requires no additional configuration.

#### 3.3.4.3 Limitations

ChromaDB is designed for single-node deployment and does not provide built-in high availability, replication, or horizontal scaling. For this project, these limitations are not relevant: the system operates as a single-user, locally-executed prototype tool. However, these architectural constraints should be acknowledged if the system were to be extended to a multi-user, production deployment scenario. Migration to a distributed vector database such as Qdrant or Milvus would be the recommended path at that stage.

#### 3.3.4.4 Role in The System

In the knowledge base construction phase, ChromaDB receives the 768-dimensional UniXcoder embeddings along with their associated metadata and persists them to local storage. During the auditing phase, the similarity_search_with_score method returns the top-k most relevant vulnerability examples for a given embedded code query. The returned documents—containing vulnerable code snippets, CWE/CVE identifiers, and incident descriptions—are formatted into the context section of the LLM prompt, providing the evidence grounding for the security audit.

### 3.3.5 LLM Model

The OpenAI GPT model serves as the "reasoning" and generation engine of the security auditor. After the retrieval pipeline delivers relevant vulnerability examples from the knowledge base, the LLM is responsible for analyzing the submitted source code against this retrieved context, identifying potential vulnerabilities, explaining their root causes, mapping findings to OWASP and CWE references, and generating actionable remediation recommendations.

The use of GPT within a RAG framework addresses a fundamental limitation of LLM-only vulnerability detection: the model's static knowledge cutoff. By grounding the LLM's analysis in external, up-to-date vulnerability knowledge retrieved at inference time, the system ensures that findings reference current CVE entries, OWASP Top 10 2025 categories, and real-world incident patterns that post-date the model's training data. This is the core architectural value of the RAG approach: the LLM provides reasoning capabilities, while the retrieval system provides current, evidence-based security knowledge.

#### 3.3.5.1 Prompt Structure

The LLM is accessed via the OpenAI API, configured with a system prompt that establishes its role as an expert security auditor. For each analysis, the prompt template combines:

- A system instruction defining the auditor's role, required output format, and security analysis methodology.
- The retrieved context containing the top-k similar vulnerability examples with their CWE/CVE identifiers, incident descriptions, and vulnerable-secure code pairs.
- The user's source code submitted for analysis.
- Structured output requirements specifying that the response must include identified vulnerabilities, OWASP/CWE mapping, root cause analysis, exploit scenario description, and specific remediation guidance.

This template is constructed dynamically at inference time, with the retrieved context determining which vulnerability patterns guide the analysis of the submitted code.

#### 3.3.5.2 Role in The System

The GPT model is the final stage of the RAG pipeline. It receives the retrieved knowledge and the code under analysis as a combined, structured prompt, performs contextual reasoning, and outputs a structured security audit report. The API integration uses LangChain's ChatOpenAI wrapper, which provides standardized interface methods compatible with the rest of the LangChain pipeline, including streaming support and token usage tracking for cost monitoring during evaluation.

The combination of these five components: Python as the implementation language, LangChain as the orchestration layer, SecureCode v2.0 as the knowledge source, UniXcoder as the embedding model, ChromaDB as the vector store, and GPT as the “reasoning” engine and report generation forms the complete technical stack of the RAG-based security auditor. Each component was selected based on a specific set of technical and practical requirements, and the modular architecture ensures that any individual component can be upgraded or replaced as the project evolves beyond the prototype stage.

## 3.4 Project Setup Procedure

The project environment was prepared by creating a local Python-based workspace for the RAG-based security auditor prototype. First, the required project directories were organized to separate the dataset files, preprocessing outputs, vector database storage, and source code scripts. The vulnerability dataset was placed in a dedicated data directory, while the generated embeddings and metadata were stored persistently in ChromaDB to allow later retrieval without rebuilding the knowledge base each time.

After organizing the project files, the required Python libraries were installed, including the libraries needed for dataset handling, UniXcoder embedding generation, ChromaDB vector storage, LangChain pipeline orchestration, and OpenAI API communication. The OpenAI API key was configured through environment variables rather than being written directly inside the source code, in order to follow basic security practices and prevent accidental exposure of sensitive credentials.

The setup procedure was divided into two main stages. In the offline stage, the dataset was loaded, cleaned, converted into document objects, embedded using UniXcoder, and stored in ChromaDB together with the related metadata. In the online stage, user-submitted source code was embedded using the same model, compared with the stored vulnerability embeddings, and used to retrieve the most relevant examples. These retrieved examples were then combined with the user code into a structured prompt and sent to the GPT-4o model to generate the final vulnerability analysis report.


**Figure 6: Project Setup Procedure**

Figure 6 illustrates the main steps followed to prepare the development environment and execute the prototype pipeline.

## 3.5 Chapter Summary

This chapter described the development environment and technical setup used to build the proposed RAG-based security auditor prototype. It presented the hardware and software environment, as well as the main tools and libraries used in the system, including Python, LangChain, UniXcoder, ChromaDB, and the OpenAI GPT-4o model. Each component was selected to support a specific part of the RAG pipeline, from dataset preparation and embedding generation to semantic retrieval and LLM-based analysis.

The chapter also explained the general setup procedure followed to prepare the project workspace and execute the system pipeline. The environment was designed to be lightweight and practical, avoiding the need for local fine-tuning or heavy model training. This setup provides the technical foundation for the next chapter, which presents the proposed system implementation with an overview of the workflow and explains how the knowledge base construction and vulnerability analysis pipeline are carried out in practice.

# Chapter 4: Practical Implementation

## 4.1 Introduction

This chapter presents the practical realization of the RAG-based security auditor. Building upon the theoretical foundations and the environment setup described in the previous chapters, the implementation translates the architectural design into a functioning pipeline that ingests a curated vulnerability knowledge base, retrieves security-relevant context for user-submitted source code, and generates structured audit reports grounded in that context. The system is implemented in Python using the LangChain orchestration framework, with UniXcoder providing vector embeddings and ChromaDB serving as the vector store. The Open AI GPT-4 model acts as the "reasoning" and report-generation engine.

The chapter is organised around the distinct stages of the pipeline: the ingestion pipeline that constructs the vector knowledge base from SecureCode v2.0; the retrieval pipeline that identifies relevant vulnerability examples for a given piece of code; and the LLM layer that synthesises this information into a comprehensive security audit. The system architecture is first described to provide an overall view of the data flow, followed by detailed walkthroughs of each stage. The chapter concludes with the evaluation methodology, a discussion of results, potential future enhancements, and a summary of the implementation's contributions.

## 4.2 System Architecture

The implemented system follows a modular Retrieval-Augmented Generation architecture designed for code-snippet-level vulnerability auditing. The system is divided into three main stages: offline knowledge base construction, online retrieval, and LLM-based security analysis. Each stage is implemented as a separate Python module, allowing the system to remain simple, maintainable, and easy to extend in later development phases.

The current prototype does not perform repository-level scanning or AST-aware chunking. Instead, each vulnerable code sample from the dataset is treated as one document in the knowledge base. Similarly, the submitted input code is embedded as a single query string during retrieval. This design was selected for the first implementation phase in order to keep the system lightweight and suitable for evaluating the effect of RAG on CWE classification and vulnerability explanation.

The architecture consists of the following components:

- Configuration Layer.
  The config.py file defines the main system paths and constants, including the dataset location, ChromaDB persistence directory, Chroma collection name, UniXcoder model name, number of retrieved examples, and OpenAI model name. Centralizing these values makes it easier to modify the system configuration without changing the core implementation.
- Embedding Layer.
  The embeddings_unixcoder.py module implements a custom LangChain-compatible embedding class using microsoft/unixcoder-base. This module loads the UniXcoder tokenizer and model, detects whether CUDA is available, embeds documents and queries in batches, extracts the [CLS] token representation, and normalizes the embedding

vectors. The same embedding function is used for both knowledge base documents and user-submitted query code to ensure that both are represented in the same semantic vector space.

- Ingestion Layer.
  The ingestion.py module builds the vector knowledge base. It reads the SecureCode-derived CSV file, validates the required columns, removes rows without vulnerable code, converts each vulnerable code snippet into a LangChain Document, and attaches security metadata such as CWE ID, OWASP category, programming language, CVE ID, CVSS score, and incident description. These documents are embedded using UniXcoder and stored in a persistent ChromaDB collection. During rebuilding, the system removes existing records from the collection before inserting the new documents, ensuring that the vector store reflects the current dataset version.
- Retrieval Layer.
  The retrieval.py module loads the persistent ChromaDB collection and embeds the submitted code using the same UniXcoder embedding function. It then retrieves the top-k most semantically similar vulnerable code examples using ChromaDB similarity search with relevance scores. In the current implementation, the default value of k is 3. The retrieved examples are returned together with their metadata and similarity scores.
- LLM Analysis Layer.
  The llm_layer.py module constructs the final prompt for the LLM. It formats the retrieved examples into a context block containing similarity score, example ID, CWE, OWASP category, programming language, CVE ID, CVSS score, incident description, and vulnerable code. This retrieved context is combined with the submitted input code and sent to the OpenAI model through LangChain's ChatOpenAI wrapper. The model is instructed to return a structured JSON response that includes the vulnerability verdict, confidence score, predicted vulnerability type, CWE/OWASP/CVE alignment, retrieval evidence, risk explanation, remediation, and limitations.

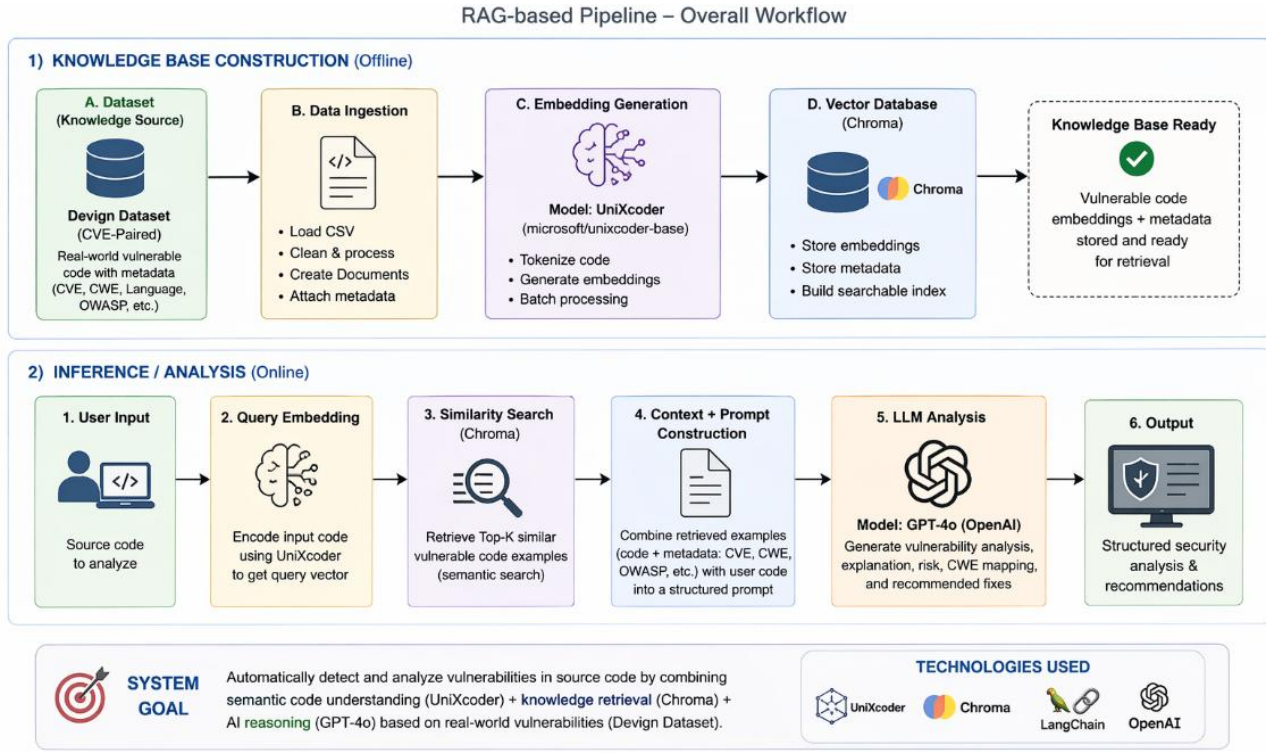


**Figure 7: Overall Workflow**

The design in Figure 7 separates the system into independent stages. The ingestion stage can be rerun whenever the knowledge base is updated, while the retrieval and LLM analysis stages operate at runtime for each submitted code snippet. This modularity also supports future enhancements, such as directory-level CLI scanning, code chunking, secure-code retrieval, CVE description enrichment, metadata filtering, and semantic description comparison.

## 4.3 Ingestion Pipeline

The ingestion pipeline is executed offline to construct the vector knowledge base used by the RAG-based security auditor. In the current implementation, the pipeline reads a prepared SecureCode-derived CSV file, converts each vulnerable code sample into a LangChain document, generates semantic code embeddings using UniXcoder, and stores the resulting documents and metadata in a persistent ChromaDB collection.

The ingestion phase is separated from the runtime retrieval phase. This means that the knowledge base does not need to be rebuilt every time the user submits code for analysis. Instead, the ingestion script is executed when the dataset is first prepared or whenever the knowledge base is updated.

### 4.3.1 Data Extraction and Preprocessing

The ingestion process starts from the prepared CSV file vulnerable_code_combined.csv, which is defined in config.py through the DATA_PATH variable. This CSV file contains vulnerable code examples and their associated security metadata. The required columns are:

- example_id
- cwe_id
- owasp_category
- language
- cve_id
- cvss
- incident_description
- vulnerable_code

The ingestion script first loads the dataset using `pandas.read_csv()`. It then checks whether all required columns exist. If any required column is missing, the script raises an error to prevent building an incomplete or inconsistent knowledge base. This validation step is important because the retrieval and LLM layers depend on these metadata fields when constructing the retrieved context.

After validation, the script removes rows where the `vulnerable_code` field is empty. The remaining vulnerable code values are converted to strings to ensure consistent processing before embedding. In the current prototype, each row is treated as one complete document. No AST-aware chunking, function-level splitting, or repository-level parsing is applied during this phase.

For each valid row, a LangChain `Document` object is created. The `page_content` field contains the vulnerable code snippet, while the metadata dictionary stores the security-related information associated with that snippet. The metadata includes the example ID, CWE identifier, OWASP category, programming language, CVE identifier, CVSS score, and incident description.

### 4.3.2 Embedding Generation With UniXCoder

After the document objects are created, each vulnerable code snippet is converted into a dense vector representation using the microsoft/unixcoder-base model. UniXcoder is used because it is a code-specific pretrained model designed to represent programming language semantics more effectively than general-purpose text embeddings.

The embedding logic is implemented in embeddings_unixcoder.py through a custom class named UniXcoderEmbeddings, which inherits from LangChain's Embeddings interface. This allows the custom embedding model to be passed directly to ChromaDB through LangChain.

The model is loaded using Hugging Face's AutoTokenizer and AutoModel. During initialization, the system checks whether CUDA is available. If a GPU is available, the model is loaded on

CUDA; otherwise, it runs on CPU. This makes the implementation portable across different hardware environments.

The current implementation uses a maximum token length of 512. Longer code snippets are truncated by the tokenizer. This is acceptable for the current prototype because most evaluation samples are code snippets rather than full repositories. However, it is also a limitation of the current version, and long-code chunking is planned as a future enhancement.

The embedding class supports both document embedding and query embedding. Documents are embedded in batches to improve efficiency. In each batch, the tokenizer pads and truncates the code snippets, converts them into PyTorch tensors, and passes them through the UniXcoder model. The first token embedding, corresponding to the [CLS] representation, is used as the vector representation of the entire code snippet. The resulting vector is then normalized using L2 normalization before being stored or used for retrieval.

This embedding process allows ChromaDB to compare code snippets using semantic similarity rather than exact keyword matching.

### 4.3.3 Vector Database Storage in ChromaDB

The generated embeddings and their associated documents are stored in a persistent ChromaDB collection. The collection name is defined in config.py as:

COLLECTION_NAME = "securecode_unixcoder"

The vector database directory is also defined in config.py using the CHROMA_DIR variable. This allows the ChromaDB collection to persist locally after ingestion, so it can be reused by the retrieval pipeline without rebuilding the embeddings each time.

The metadata stored with each document includes:

**Table 2: Metadata**

| Metadata Field | Purpose |
|---|---|
| example_id | Identifies the original dataset example |
| cwe_id | Stores the CWE weakness category |
| owasp_category | Maps the example to an OWASP category |
| language | Indicates the programming language of the code |
| cve_id | Stores the related CVE identifier when available |
| cvss | Stores the severity score when available |

| | |
|---|---|
| incident_description | Provides real-world context for the vulnerability |

After the documents are inserted, the ChromaDB collection becomes ready for similarity search during runtime. The retrieval module later loads the same collection and uses it to retrieve the top-k most semantically similar vulnerable code examples for a submitted code snippet.

This storage design keeps the current prototype simple and lightweight while still supporting evidence-grounded vulnerability analysis through retrieved code examples and metadata.

## 4.4 Retrieval Pipeline

The retrieval pipeline is executed at runtime when a code snippet is submitted for analysis. Its purpose is to retrieve vulnerable code examples from the ChromaDB knowledge base that are semantically similar to the submitted input code. These retrieved examples are later passed to the LLM as supporting evidence.

Unlike the ingestion pipeline, which is executed offline to build the vector database, the retrieval pipeline is executed for each audit request. The current implementation does not perform language detection, metadata filtering, AST-aware chunking, or Maximal Marginal Relevance retrieval. Instead, it embeds the submitted code directly using the same UniXcoder embedding function used during ingestion, then performs a top-k semantic similarity search against the stored ChromaDB collection.

### 4.4.1 Input Handling and Preprocessing

The input to the retrieval pipeline is a source code snippet represented as a string. In the current prototype, the system is designed for code-snippet-level or function-level auditing rather than full repository-level analysis. Therefore, the submitted code is used directly as the query text.

If the submitted input is long, it is still passed directly to the embedding model, where it may be truncated according to the tokenizer's maximum sequence length. This behavior is acceptable for the current evaluation phase, where the test samples are individual vulnerable code snippets. However, handling long files and directory-level scanning is planned as a future enhancement.

The retrieval function receives the query code and the number of results to return:

#def retrieve_similar_code(query_code: str, k: int = TOP_K):

The default value of TOP_K is defined in config.py as:

#TOP_K = 3

This means that, for each submitted code snippet, the system retrieves the three most semantically similar vulnerable examples from the knowledge base

### 4.4.2 Query Embedding and Similarity Search

Before performing retrieval, the system loads the persistent ChromaDB collection through the load_vectorstore() function. This function initializes the same UniXcoder embedding function used during ingestion and connects it to the previously stored ChromaDB collection.

Using the same embedding model during both ingestion and retrieval is essential because both stored documents and submitted query code must exist in the same vector space. If different embedding models were used, similarity scores would not be meaningful.

After loading the vector store, the submitted code is embedded by the UniXcoder embedding function and compared against the stored document embeddings in ChromaDB. The retrieval process uses LangChain's similarity_search_with_relevance_scores() method, this method returns a list of retrieved results. Each result consists of (Document, relevance_score).

The Document contains the retrieved vulnerable code in its page_content field and the associated metadata in its metadata field. The relevance score indicates how similar the submitted code is to the retrieved example according to ChromaDB's similarity calculation.

The current implementation uses semantic similarity retrieval only. It does not use Maximal Marginal Relevance (max_marginal_relevance_search) and does not apply metadata filters such as programming language or CWE category. This design keeps the first prototype simple and makes it easier to evaluate the direct effect of code-semantic retrieval on CWE classification.

### 4.4.3 Results Post-processing

Each retrieved result carries the raw code chunk, CWE/CVE identifiers, OWASP category, incident description, and a relevance score (cosine similarity score). These are assembled into a context block for the LLM.

After retrieval, the results are formatted into a readable context block by the build_context() function in llm_layer.py. For each retrieved example, the context includes the similarity score, example ID, CWE, OWASP category, programming language, CVE ID, CVSS score, incident description, and vulnerable code.

This formatted context is then inserted into the LLM prompt together with the submitted code. As a result, the model can compare the input code with known vulnerable examples and produce a more evidence-grounded analysis.

This retrieval design provides the central mechanism of the RAG approach: instead of relying only on the LLM's internal knowledge, the system augments the model with similar vulnerability examples from the local knowledge base before generating the final audit output.

## 4.5 LLM Layer

The LLM layer is responsible for interpreting the submitted code together with the retrieved vulnerability examples and producing a structured security analysis. In the current implementation, the LLM layer is implemented in llm_layer.py and uses LangChain's ChatOpenAI wrapper to communicate with the OpenAI model defined in config.py.

This layer has two main purposes. First, it supports the normal RAG-based auditing workflow, where the model receives both the submitted code and the retrieved SecureCode examples. Second, it supports a baseline LLM-only workflow used during evaluation, where the same model analyzes the submitted code without any retrieved context. This allows the project to compare the effect of retrieval augmentation against direct LLM prompting.

### 4.5.1 Model Selection and Configuration

The OpenAI GPT model is used as the reasoning and analysis component of the system. The specific model name is defined in config.py using the OPENAI_MODEL variable and the model is accessed through LangChain's ChatOpenAI wrapper.

The temperature is set to 0 to encourage more deterministic and consistent outputs. This is important for security auditing and evaluation because the system is expected to produce stable predictions when analyzing the same input code. A high temperature could increase variation in predicted CWE values and make evaluation less reliable.

The OpenAI API key is loaded from environment variables using the dotenv package. This avoids hard-coding sensitive credentials inside the source code and follows basic secure development practice.

### 4.5.2 Prompt Design

Before the prompt is sent to the LLM, the retrieved examples are converted into a structured text block using the build_context() function. This function receives the top-k retrieved results from the retrieval pipeline, where each result contains a LangChain Document object and a relevance score.

For every retrieved example, the context block includes:

**Table 3: Context block**

| Field | Description |
|---|---|
| Similarity Score | Relevance score returned by ChromaDB |
| Example ID | Identifier of the retrieved dataset example |
| CWE | CWE category associated with the retrieved code |
| OWASP | OWASP Top 10 category associated with the retrieved code |

| | |
|---|---|
| Language | Programming language of the retrieved code |
| CVE | Related CVE identifier when available |
| CVSS | Severity score when available |
| Incident | Natural-language incident description |
| Code | Retrieved vulnerable code snippet |

This context is then inserted into the RAG prompt. The purpose of this step is to make the LLM's analysis evidence-grounded instead of relying only on its internal knowledge.

The current implementation uses direct prompt strings rather than a separate ChatPromptTemplate. Two prompt variants are implemented:

1. Base LLM prompt, used in explain_vulnerability_base().
2. RAG prompt, used in explain_vulnerability() or explain_vulnerability_rag().

The base prompt sends only the submitted code to the LLM. It asks the model to analyze the code, determine whether it is vulnerable, and return a structured JSON response. This prompt is used as the baseline during evaluation.

The RAG prompt sends both the submitted code and the retrieved SecureCode examples. The model is instructed to use retrieved examples as supporting evidence but not to blindly copy their labels. This distinction is important because retrieved examples may be semantically similar but not always identical in vulnerability behavior.

The RAG prompt also includes rules such as:

- Base the final verdict on the input code itself.
- Use retrieved examples only as supporting evidence.
- Do not invent CVE IDs.
- If the retrieved examples are weakly related, explain that limitation.
- Return valid JSON only.

These instructions are designed to reduce hallucination and make the output easier to parse automatically.

### 4.5.3 Report Generation

The current implementation requires the LLM to return valid JSON. This design was selected because JSON output is easier to evaluate programmatically, especially when comparing the Base LLM and RAG approaches.

The most important field for the current evaluation is cwe_id, which is extracted from the first item in the vulnerability_types list. This predicted CWE is compared against the true CWE label from the evaluation dataset.

The JSON format also supports future improvements because the same structure can be used to store audit results, compare outputs, generate reports, or display findings in a CLI or web interface.

## 4.8 Conclusion

This chapter presented the practical implementation of the RAG-based security auditor. The implemented system follows a modular pipeline composed of an ingestion layer, a retrieval layer, and an LLM analysis layer. The ingestion layer reads a SecureCode-derived CSV dataset, converts vulnerable code examples into LangChain documents, generates UniXcoder embeddings, and stores them with metadata in a persistent ChromaDB collection. The retrieval layer embeds submitted code snippets and retrieves the top-k semantically similar vulnerable examples. The LLM layer combines the retrieved evidence with the submitted code and produces structured JSON output containing the vulnerability verdict, CWE/OWASP mapping, risk explanation, remediation guidance, and limitations.

The evaluation compared two approaches: a base LLM that analyzes code without retrieval and a RAG-based LLM that receives retrieved vulnerability examples as additional context. Using a held-out test set of 50 samples from the top five CWE categories, the RAG-based approach improved the average F1-score from 0.434 to 0.793. The results suggest that retrieval augmentation helps the model classify vulnerable code into the correct CWE category more reliably than using the LLM alone.

At the same time, the implementation remains a prototype. It currently focuses on snippet-level auditing and CWE classification, and it does not yet support full repository scanning, safe-versus-vulnerable detection, long-code chunking, or semantic comparison using secure-code descriptions. These limitations define the next development phase of the project.

Overall, the implemented system demonstrates the feasibility of using Retrieval-Augmented Generation for evidence-grounded security auditing. By combining code-specific embeddings, vector-based retrieval, vulnerability metadata, and structured LLM output, the project establishes a practical foundation for a more advanced AI-based security auditing tool in future work.

# Chapter 5: Evaluation and Results

## 5.1 Introduction

This chapter presents the evaluation methodology and results of the implemented RAG-based security auditor. The purpose of this evaluation is to measure whether retrieval augmentation improves the model's ability to classify vulnerable code into the correct CWE category when compared with using the LLM alone.

The evaluation focuses on CWE classification rather than full safe-versus-vulnerable detection. This is because the current knowledge base is constructed mainly from vulnerable SecureCode examples. Therefore, the evaluation is designed to answer the following question:

Q: Does retrieving semantically similar vulnerable code examples improve the LLM's ability to predict the correct CWE category?

To answer this question, two approaches were compared:

1. Base LLM: The submitted code is sent directly to the LLM without retrieval context.
2. RAG-based LLM**:** The submitted code is first used to retrieve semantically similar vulnerable examples from ChromaDB, and the retrieved examples are then included in the LLM prompt.

Both approaches use the same OpenAI model and the same JSON output format. The only difference is that the RAG-based approach receives retrieved vulnerability evidence from the knowledge base.

## 5.2 Evaluation Dataset Preparation

To evaluate the system, a held-out test set was constructed from the SecureCode-derived dataset. First, the dataset was analyzed to identify the most frequent CWE categories. The top five CWE categories were selected because they had enough samples to support a small but meaningful evaluation.

The selected CWE categories were: CWE-16, CWE-284, CWE-287, CWE-502, CWE-89. For each CWE category, 10 samples were selected, resulting in a test set of 50 vulnerable code samples: 5 CWE categories × 10 samples per CWE = 50 test samples

To avoid retrieval leakage, the selected test samples were removed from the knowledge base dataset before rebuilding the ChromaDB vector collection. This step is important because if the test samples remained inside the knowledge base, the RAG system might retrieve the exact same code sample during evaluation, which would make the results unfairly high.

After removing the test samples, the remaining dataset was used to rebuild the ChromaDB knowledge base. The test set was then used only for evaluation.

## 5.3 Evaluation Methodology

Each sample in the test set contains a vulnerable code snippet and a ground-truth CWE label taken from the dataset. For each sample, the system executed two prediction workflows.

1- Base LLM workflow: In the base workflow, the vulnerable code was sent directly to the LLM. The model was asked to analyze the code and return a JSON response containing the predicted CWE.
2- RAG Workflow: In the RAG workflow, the vulnerable code was first passed to the retrieval pipeline. The system embedded the code using UniXcoder and retrieved the top three semantically similar vulnerable examples from the ChromaDB knowledge base. These retrieved examples, along with their CWE, OWASP, CVE, CVSS, and incident metadata, were then inserted into the LLM prompt.

The predicted CWE from each workflow was compared with the ground-truth CWE label from the dataset.

## 5.4 Evaluation Metrics

The evaluation uses four standard classification metrics:

- Accuracy
- Precision
- Recall
- F1 Score

The metrics were computed for each CWE where: Accuracy measures how often the system correctly separates a given CWE from the other categories. Precision measures how often the system is correct when it predicts a specific CWE. Recall measures how many true samples of a CWE were correctly identified. F1-score provides a balanced measure between precision and recall.

## 5.5 Base LLM Results

Table 5 shows the performance of the base LLM without retrieval augmentation.

**Table 4: LLM performance**

| CWE | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| CWE-16 | 0.800 | 0.000 | 0.000 | 0.000 |
| CWE-284 | 0.800 | 0.000 | 0.000 | 0.000 |
| CWE-287 | 0.840 | 1.000 | 0.200 | 0.333 |

| | | | | |
|---|---|---|---|---|
| CWE-502 | 0.960 | 1.000 | 0.800 | 0.889 |
| CWE-89 | 0.980 | 1.000 | 0.900 | 0.947 |
| All (Avg) | 0.876 | 0.600 | 0.380 | 0.434 |

The base LLM achieved an average accuracy of 0.876. However, its average recall and F1-score were much lower, reaching only 0.380 and 0.434 respectively. This shows that accuracy alone is not enough to evaluate the system, because the model performed well on some categories but completely failed on others.

In particular, the base LLM failed to correctly identify any samples from CWE-16 and CWE-284, resulting in recall and F1-score values of 0.000 for both categories. This indicates that the base model struggled with these vulnerability classes when it relied only on its internal knowledge and the submitted code.

On the other hand, the base model performed strongly on CWE-502 and CWE-89, achieving F1-scores of 0.889 and 0.947 respectively. This suggests that some vulnerability patterns, such as deserialization issues and SQL injection, may be more recognizable to the model without external retrieval.

## 5.6 RAG-Based Results

Table 6 shows the performance of the RAG-based approach.

**Table 5: RAG-based approach performance**

| CWE | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| CWE-16 | 0.900 | 0.857 | 0.600 | 0.706 |
| CWE-284 | 0.940 | 1.000 | 0.700 | 0.824 |
| CWE-287 | 0.900 | 1.000 | 0.500 | 0.667 |
| CWE-502 | 0.980 | 1.000 | 0.900 | 0.947 |
| CWE-89 | 0.940 | 1.000 | 0.700 | 0.824 |
| All (Avg) | 0.932 | 0.971 | 0.680 | 0.793 |

The RAG-based system achieved an average accuracy of 0.932, average precision of 0.971, average recall of 0.680, and average F1-score of 0.793. Compared with the base LLM, the RAG approach produced a clear improvement across most metrics.

The most significant improvements appeared in the CWE categories that the base LLM failed to identify. For CWE-16, the F1-score improved from 0.000 to 0.706. For CWE-284, the F1-score improved from 0.000 to 0.824. This indicates that retrieved vulnerability examples helped the LLM recognize categories that it could not classify correctly when operating without external context.

The RAG approach also improved CWE-287, increasing its F1-score from 0.333 to 0.667. For CWE-502, the RAG system slightly improved the already strong base performance. For CWE-89, the base model achieved a higher F1-score than RAG, but the RAG result remained strong with an F1-score of 0.824. This decrease may occur when retrieved examples introduce related but not perfectly matching vulnerability context, which can slightly influence the model's classification.

## 5.7 Comparative Analysis

Table 7 summarizes the average performance of the two approaches.

**Table 6: Average Performance Comparison Between Base LLM and RAG**

| Approach | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| Base LLM | 0.876 | 0.600 | 0.380 | 0.434 |
| RAG | 0.932 | 0.971 | 0.680 | 0.793 |
| Improvement | +0.056 | +0.371 | +0.300 | +0.359 |

The RAG-based approach improved average F1-score from 0.434 to 0.793. This is the most important improvement because F1-score balances precision and recall. The improvement shows that the retrieved examples helped the model classify vulnerability categories more reliably.

The average recall increased from 0.380 to 0.680, meaning that the RAG system correctly identified more true samples of each CWE category. This is important in vulnerability detection because missing a vulnerability category can lead to incomplete or misleading security analysis.

The average precision also increased from 0.600 to 0.971. This indicates that when the RAG system predicted a CWE category, it was usually correct. Therefore, retrieval augmentation improved not only the system's ability to detect more relevant categories, but also the reliability of its predictions.

## 5.8 Discussion of Results

The evaluation results suggest that retrieval augmentation improves the LLM's ability to classify vulnerable code according to CWE categories. The improvement is especially visible for CWE classes where the base model had weak performance.

The base LLM achieved high accuracy for some categories even when recall was low. This happens because the evaluation uses a one-vs-rest approach. For example, each CWE has only 10 positive samples and 40 negative samples. If the model never predicts a certain CWE, it can still achieve 0.800 accuracy by correctly treating the 40 other samples as not belonging to that CWE. However, recall and F1-score reveal that the model completely missed the positive samples. This explains why CWE-16 and CWE-284 had 0.800 accuracy but 0.000 recall and F1-score in the base model results.

The RAG system reduced this problem by providing retrieved examples from the knowledge base. These examples gave the model additional context about vulnerability patterns, CWE mappings, and related incident descriptions. As a result, the model was better able to classify categories that were not recognized by the base LLM alone.

Overall, the results support the main assumption of this project: a RAG-based security auditor can improve code vulnerability classification by grounding the LLM's analysis in semantically similar vulnerable examples.

## 5.9 Evaluation Limitations

Although the results are promising, the evaluation has several limitations.

First, the test set is small, containing only 50 samples across five CWE categories. This is sufficient for a prototype-level evaluation, but a larger test set would be needed to make stronger conclusions about general performance.

Second, the evaluation measures CWE classification only. It does not fully measure safe-versus-vulnerable detection because the current dataset and knowledge base mainly contain vulnerable examples. Therefore, the results should be interpreted as measuring how well the system predicts the correct CWE category for vulnerable code, not as a complete vulnerability detection benchmark.

Third, the current implementation embeds each vulnerable code sample as a single document. It does not yet perform chunking for long code files, repository-level analysis, interprocedural data-flow tracking, or secure-code comparison. These limitations may affect performance when analyzing large or complex real-world programs.

Fourth, the system relies on the quality and coverage of the SecureCode-derived knowledge base. If a submitted code snippet belongs to a vulnerability pattern that is not well represented in the knowledge base, retrieval quality may decrease and the LLM's analysis may become less accurate.

Despite these limitations, the evaluation provides useful evidence that retrieval augmentation improves CWE classification in the current prototype and supports further development of the project.

## 5.10 Future Enhancements

Although the current prototype demonstrates that retrieval augmentation can improve CWE classification, several enhancements are planned to make the system more practical, scalable, and suitable for real-world security auditing.

### 5.10.1 Directory-Level CLI Auditing Tool

The current implementation analyzes a submitted code snippet as a single input. A major future enhancement is to develop a command-line interface (CLI) tool that accepts a project directory as input, automatically discovers source code files, and audits them individually.

This enhancement would make the system more useful for developers because it would move the prototype from isolated code-snippet analysis toward practical project-level auditing. The CLI tool could also support options such as selecting programming languages, limiting file size, excluding directories such as node_modules or .git, and exporting results to structured files.

### 5.10.2 Improved Knowledge Base Construction

The current knowledge base is built mainly from vulnerable code examples and metadata such as CWE, OWASP category, CVE ID, CVSS score, language, and incident description. A future version should enrich this knowledge base by extracting additional fields from SecureCode v2.0, especially:

- secure code implementation
- code descriptions
- attack demonstration
- defense-in-depth recommendations
- CVE descriptions

Including secure code examples would allow the system to compare vulnerable and secure implementations, which is important for distinguishing vulnerable code from patched or safe code. Adding CVE descriptions would also improve the knowledge available to the LLM, allowing it to reason not only from similar code but also from real-world vulnerability context.

Another planned improvement is to remove or separately handle rows that do not contain CVE IDs. This could make CVE-based enrichment easier and improve the consistency of the knowledge base during early development.

### 5.10.3 Semantic Description-Based Retrieval

The current retrieval process compares the submitted code directly with vulnerable code examples using UniXcoder embeddings. A future enhancement is to compare semantic descriptions rather than only raw code.

In this approach, the submitted code could first be summarized by an LLM or transformed using a structured representation such as SCALE. The generated description would capture information such as:

- code purpose
- input sources
- sensitive operations
- data flow behavior
- security-relevant actions
- possible vulnerability triggers

This description could then be compared with descriptions of known vulnerable code and related CVE records in the knowledge base. This would move the system closer to knowledge-level RAG, where retrieval is based on vulnerability behavior and root causes rather than only code similarity.

### 5.10.4 Long-Code Chunking Strategy

The current implementation embeds each submitted code sample as a single query. This is suitable for short snippets and function-level examples, but it is not enough for long files or full applications. UniXcoder uses a maximum token length, so long inputs may be truncated before embedding.

A future version should implement chunking strategies to handle long code files. Possible approaches include:

- function-level chunking
- class-level chunking
- sliding-window chunking
- language-aware chunking
- AST-aware chunking

The goal is to preserve security-relevant context while keeping each chunk small enough for reliable embedding and LLM analysis. This would also support directory-level auditing because real projects usually contain long files and multiple functions.

### 5.10.5 Metadata Filtering and Hybrid Retrieval

The current retrieval pipeline uses semantic similarity search only. Future versions can improve retrieval precision by using metadata filters such as:

- programming language
- CWE category
- OWASP category
- CVE availability

- CVSS severity

For example, if the submitted file is Python code, the system could prioritize Python examples in the knowledge base. If the system detects possible SQL-related behavior, it could prioritize injection-related examples.

Another future improvement is hybrid retrieval, combining semantic search with lexical search. Semantic retrieval is useful for finding conceptually similar code, while lexical search is useful for exact terms such as function names, CVE identifiers, library names, and dangerous API calls. Combining both approaches may improve retrieval quality.

### 5.10.6 Safer and More Reliable Output Validation

The current system instructs the LLM to return structured JSON and uses a parser to extract the response. Future versions can improve output reliability by enforcing stricter schemas and adding validation logic.

For example, the system could check whether:

- the predicted CWE exists in the retrieved evidence
- the CVE ID was actually present in the retrieved examples
- the JSON contains all required fields
- the confidence score is within a valid range
- the remediation is consistent with the detected vulnerability

A lightweight validator agent could also be added to review the first LLM response and reject unsupported vulnerability claims. This would help reduce hallucinations and improve trustworthiness.

### 5.10.7 Expanded Evaluation

The current evaluation focuses on CWE classification for 50 held-out vulnerable samples from the top five CWE categories. Future evaluation should be expanded in several ways.

First, the test set should include more CWE categories and more samples per category. Second, the evaluation should include safe or patched code examples so that the system can be tested on full vulnerable-versus-safe detection. Third, future experiments should evaluate OWASP mapping accuracy, CVE alignment accuracy, retrieval quality, and remediation quality.

# Conclusion:

This project discussed the recent state-of-the-art approaches in the realm of vulnerability detection using large language models, starting with a foundation for the theoretical concepts of these approaches, moving forward to presenting the practical implementation of the RAG-based security auditor. The implemented system follows a modular pipeline composed of an ingestion layer, a retrieval layer, and an LLM analysis layer. The ingestion layer reads a SecureCode-derived CSV dataset, converts vulnerable code examples into LangChain documents, generates UniXcoder embeddings, and stores them with metadata in a persistent ChromaDB collection.

The retrieval layer embeds submitted code snippets and retrieves the top-k semantically similar vulnerable examples. The LLM layer combines the retrieved evidence with the submitted code and produces structured JSON output containing the vulnerability verdict, CWE/OWASP mapping, risk explanation, remediation guidance, and limitations.

The evaluation compared two approaches: a base LLM that analyzes code without retrieval and a RAG-based LLM that receives retrieved vulnerability examples as additional context. Using a held-out test set of 50 samples from the top five CWE categories, the RAG-based approach improved the average F1-score from 0.434 to 0.793. The results suggest that retrieval augmentation helps the model classify vulnerable code into the correct CWE category more reliably than using the LLM alone.

At the same time, the implementation remains a prototype. It currently focuses on snippet-level auditing and CWE classification, and it does not yet support full repository scanning, safe-versus-vulnerable detection, long-code chunking, or semantic comparison using secure-code descriptions. These limitations define the next development phase of the project.

Overall, the implemented system demonstrates the feasibility of using Retrieval-Augmented Generation for evidence-grounded security auditing. By combining code-specific embeddings, vector-based retrieval, vulnerability metadata, and structured LLM output, the project establishes a practical foundation for a more advanced AI-based security auditing tool in future work.

# References


[1] M. M. A. A. Md Hasan Saju, "An Empirical Evaluation of LLM-Based Approaches for Code Vulnerability Detection: RAG, SFT, and Dual-Agent Systems," *arXiv,* p. 6, 2026.

[2] J. X. Y. P. C. Y. C. a. X. J. Zihan Wu, "MulVul: Retrieval-augmented Multi-Agent Code Vulnerability Detection via Cross-Model Prompt Evolution," arXiv, 2026.

[3] Y. L. X. S. Di Cao1, "RealVul: Can We Detect Vulnerabilities in Web Applications with LLM?," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024)*, 2024.

[4] J. G. 1. C. 1. X. X. 1. Z. S. 1. X. Z. 1, "REPOAUDIT: An Autonomous LLM-Agent for Repository-Level Code Auditing," in *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, Vancouver, Canada, 2025.

[5] G. Z. K. W. Y. Z. Y. W. W. D. J. F. M. L. B. C. X. P. T. M. Y. L. XUEYING DU, "Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG," *ACM Transactions on Software Engineering and Methodology,* p. 26, 2026.

[6] W. C. S. S. M. Y. Minjae Seo∗, "AutoPatch: Multi-Agent Framework for Patching Real-World CVEs Generated by Outdated LLMs," arXiv, 2025.

[7] X. C. ,. L. ,. X. C. G. Zhijie Chen, "ReVul-CoT: Towards Effective Software Vulnerability Assessment with Retrieval-Augmented Generation and Chain-of-Thought Prompting," arXiv, China, 2025.

[8] Y. Huang, "Fine-tuning of Large Language Models for Domain-Specific Cybersecurity Knowledge," *arXiv,* 2025.

[9] J. K. T. A. R. C. L. A. S. &. P. T. Samuele Pasini, "Evaluating and improving the robustness of security attack detectors generated by LLMs," *Empir Software Eng,* pp. 31-35, 2026.

[10] S. Thornton, "SecureCode v2.0: A Production-Grade Dataset for Training Security-Aware Code Generation Models," *arXiv,* 2025.

[11] A. S. H. M. E. K. A. F. D. V. B. S. B. P. C. A. O. Xavier Cadet, "Retrieval-Augmented LLMs for Security Incident Analysis," *arXiv,* 2026.

[12] V. S. N. S. K. A. C. T. S. Idan Habler, "Adversarial Hubness Detector: Detecting Hubness Poisoning in Retrieval-Augmented Generation Systems," *arxiv,* 2026.

[13] S. Hu, "RAG-Shield: Security Framework for Retrieval-Augmented Generation," 2025. [Online]. Available: https://github.com/SidereusHu/RAG-Shield.

[14] S. L. N. D. Y. W. M. Z. J. Y. Daya Guo, "UniXcoder: Unified Cross-Modal Pre-training for Code Representation," *arXiv,* 2022.

[15] V. V. C. A. S. A. C. Charugin, "Analysis of software code preprocessing methods to improve the effectiveness of using large language models in vulnerability detection tasks," *Computational nanotechnology,* vol. 12.3, pp. 67-79, 2025.

[16] J. C. D. P. S. L. X. W. Weiliang Qi, "Enhancing Pre-Trained Language Models for Vulnerability Detection via Semantic-Preserving Data Augmentation," *arXiv,* 2024.

[17] H. L. T. S. A. E. H. Mootez Saad, "On the Effect of Token Merging on Pre-trained Models for Code," *arXiv,* 2025.

[18] P. M. B. R. Kapparad, "Tighter Clusters, Safer Code Improving Vulnerability Detection with Enhanced Contrastive Loss," *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies,* vol. Volume 4: Student Research Workshop, pp. 247-252, 2025.

[19] T. J. Y. W. T. X. N. Z. J. L. Zhengbin Zou, "Code vulnerability detection based on augmented program dependency graph and optimized CodeBERT," *Scientific reports,* vol. 15, 2025.

[20] S. R. S. D. Mohammad Farhad, "HYDRA: A Hybrid Heuristic-Guided Deep Representation Architecture for Predicting Latent Zero-Day Vulnerabilities in Patched Functions," *arXiv,* 2026.

[21] Y. K. Z. D. Y. L. X. S. Wenxin Tao, "Adapting LLM to Generate Vulnerability Descriptions and Repair Suggestions," *2025 IEEE 5th International Conference on Software Engineering and Artificial Intelligence (SEAI),* pp. 184-188, 2025.

[22] C. a. I. E. a. S. R. Tony, "Retrieve, Refine, or Both? Using Task-Specific Guidelines for Secure Python Code Generation," *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME),* pp. 368-379, 2025.

[23] V. H. Manisha Mukherjee, "SOSecure: Safer Code Generation with RAG and StackOverflow Discussions," *arXiv,* 2025.

[24] X. W. K. Y. Yang Jiao, "PR-Attack: Coordinated Prompt-RAG Attacks on Retrieval-Augmented Generation in Large Language Models via Bilevel Optimization," *Proceedings*

*of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval,* pp. 656 - 667, 2025.

[25] S. S. C. X. M. X. Y. K.-C. W. L. Z. Gaurav Bagwe, "Your RAG is Unfair Exposing Fairness Vulnerabilities in Retrieval-Augmented Generation via Backdoor Attacks," *arXiv,* 2025.

[26] Q. X. J. H. B. X. W. W. Y. C. F. L. B. Z. H. H. K. W. Zi Liang, "Argus: Reorchestrating Static Analysis via a Multi-Agent Ensemble for Full-Chain Security Vulnerability Detection," *arXiv,* 2026.

[27] S. L. Y. Z. X. L. L. Z. Fang Liu1, "SecureReviewer: Enhancing Large Language Models for Secure Code Review through Secure-aware Fine-tuning," in *48th International Conference on Software Engineering (ICSE 2026)*, Rio de Janeiro, Brazil, 2026.

[28] F. R. John Pellew, "RealVuln Benchmarking Rule-Based, General-Purpose LLM, and Security-Specialized Scanners on Real-World Code," *arXiv,* 2026.

[29] S. S. K. K. S. T. M. Subho Halder, "Seclens: Role-specific Evaluation of LLM's for security vulnerablity detection," *arXiv,* 2026.

[30] S. G. T. Y. L. Y. F. C. W. D. M. D. Alperen Yildiz, "Benchmarking LLMs and LLM-based Agents in Practical Vulnerability Detection for Code Repositories," *Association for Computational Linguistics (ACL),* pp. 30848--30865, 2025.

[31] M. H. S. P. H. P. A. C. G. S. Saad Ullah, "LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet) A Comprehensive Evaluation, Framework, and Benchmarks," *arXiv,* 2024.

[32] L. S. J. R. G. M. B. E. H. D. B. M. B. J. C. D. Z. Nancy Lau, "ZeroDayBench Evaluating LLM Agents on Unseen Zero-Day Vulnerabilities for Cyberdefense," *arXiv,* 2026.

[33] S. K. D. K. J. S. C. Chinmay Pushkar, "Beyond Single Bugs: Benchmarking Large Language Models for Multi-Vulnerability Detection," *arXiv,* 2025.

[34] N. T. B. T. Z. C. Y. X. Z. M. W. J. J. J. C. H. H. H. H. N. C. Y. H. J. T. R. L. Y. Y. H. W. A. F. L. E. L. O. L. K. S. D. L. Yikun Li, "Out of Distribution, Out of Luck: How Well Can LLMs Trained on Vulnerability Datasets Detect Top 25 CWE Weaknesses?," *arXiv,* 2026.

[35] H. H. F. L. Q. Y. J. W. Z. L. P. L. M. W. H. Z. Y. L. Yanming Mu, "Towards Secure Retrieval-Augmented Generation: A Comprehensive Review of Threats, Defenses and Benchmarks," *arXiv,* 2026.

[36] S. Thornton, "Retrieval Pivot Attacks in Hybrid RAG: Measuring and Mitigating Amplified Leakage from Vector Seeds to Graph Expansion," *arXiv,* 2026.

[37] Z. S. *. a. V. Bilicki, "A New Approach to Web Application Security: Utilizing GPT Language Models for Source Code Inspection," *Future Internet,* p. 27, 2023.

[38] E. Smirnov, "Vector Database Comparison 2026: ChromaDB vs. Qdrant vs. pgvector vs. Pinecone vs. LanceDB for Production RAG," 4xxi Software Ltd, 2026. [Online]. Available: https://4xxi.com/articles/vector-database-comparison/. [Accessed 2026].

[39] J. P. Talaya Farasat, "Optimizing Code Embeddings and ML Classifiers for Python Source code Vulnerability Detection," *Proceedings of the IEEE/ACM 12th International Conference on Big Data Computing, Applications and Technologies,* 2025.

[40] J. Z. X. j. Y. L. J. H. Z. Z. Ziyin Zhou, "Tug-of-War within A Decade: Conflict Resolution in Vulnerability Analysis via Teacher-Guided Retrieval-Augmented Generations," *arXiv,* 2026.

[41] C. K. E. B. Alireza Lotfi, "Automated Vulnerability Validation and Verification: A Large Language Model Approach," *arXiv,* 2025.

[42] Y. W. H. X. J. S. F. Z. P. L. Z. G. Ruitong Liu, "Vul-LMGNNs: Fusing language models and online-distilled graph neural networks for code vulnerability detection," *arXiv,* 2025.

[43] N. Daryabar, "Security in Machine Learning: Exposing LLM Vulnerabilities through Poisoned Vector Databases in RAG-based System," University of Padova, Department of Mathematics “Tullio Levi-Civita", 2024.

[44] D. D. R. E. O. M. Emil Marian Pasca, "LLM-Driven, Self-Improving Framework for Security Test Automation: Leveraging Karate DSL for Augmented API Resilience," *IEEE Access,* vol. 15, pp. 56861-56886, 2025.

[45] S. W. Y. Q. L. C. X. M. Bo Lin, "Give LLMs a Security Course: Securing Retrieval-Augmented Code Generation via Knowledge Injection," *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25). Association for Computing Machinery, New York, NY, USA,* p. 3356–3370, 2025.

[46] S. T. H. A. S. K. T. H. P. S. N. M. A. H. N. A. J. Z. S. K. S. M. T. F. F. Dipayan Saha, "SV-LLM: An Agentic Approach for SoC Security Verification using Large Language Models," *arXiv,* 2025.

[47] M. P. S. W. M. S. Stefan Stein, "Leveraging MDS2 and SBOM data for LLM-assisted vulnerability analysis of medical devices," *Computational and structural biotechnology journal,* vol. 28, p. 267–280, 2025.

[48] K. Z. H. T. B. M. A. F. L. C. C. N. T. Richard A. Dubniczky, "CASTLE: Benchmarking Dataset for Static Code Analyzers and LLMs towards CWE Detection," *arXiv,* 2025.

[49] Y. F. O. I. C. S. X. C. B. A. D. W. B. R. Y. C. Yangruibo Ding, "Vulnerability Detection with Code Language Models: How Far Are We?," *arXiv,* 2024.

[50] N. S. H. J. S. H. V. P. S. W. Md Basim Uddin Ahmed, "SecVulEval: Benchmarking LLMs for Real-World C/C++ Vulnerability Detection," *arXiv,* 2025.

[51] F. P. L. V. A. M. Luca Ferrari, "The OWApp Benchmark: An OWASP-Compliant Vulnerable Android App Dataset," *2025 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW),* pp. 569-580, 2025.

[52] S. O. C. R. L. M. P. L. Lukas Ammann, "Securing RAG: A Risk Assessment and Mitigation Framework," *arXiv,* 2025.

[53] S. R. Praveen Chamarti, "Guidance for Securing Sensitive Data in RAG Applications using Amazon Bedrock," June 2025. [Online]. Available: https://aws-solutions-library-samples.github.io/ai-ml/securing-sensitive-data-in-rag-applications-using-amazon-bedrock.html. [Accessed April 2026].

[54] V. V. Mann Vipul, "Retrosynthesis Prediction using Grammar-based Neural Machine Translation: An Information-Theoretic Approach," *chemrxiv,* 2021.